



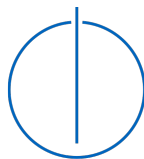
TUM SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Adaptivity between ACID and BASE in Concurrent Updates

Mykhailo Kazymyr





**TUM SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY**

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Adaptivity between ACID and BASE in Concurrent Updates

Adaptivität zwischen ACID und BASE bei nebenläufigen Aktualisierungen

Author:	Mykhailo Kazymyr
Supervisor:	Prof. Dr. Maribel Acosta
Advisor:	M.Sc. Zenon Zacouris
Submission Date:	15.09.2026



I confirm that this bachelor's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 15.09.2026

Mykhailo Kazymyr

Acknowledgments

I would like to express my deep gratitude to my advisor, Zenon Zacouris, and to Prof. Dr. Maribel Acosta and the Chair of Data Engineering for giving me the opportunity to work on such an interesting and impactful topic. I am thankful to Zenon for the insights and knowledge he shared with me throughout this project, as our discussions helped me find a clear direction whenever the work became difficult, and his feedback made every chapter of this thesis better.

Working on this thesis was one of the most rewarding experiences of my studies and allowed me to grow my skills and develop my own ideas. I also sincerely appreciate the support of the chair, which gave me access to the resources I needed to run the experiments and to the expertise that guided me through them, and without which this thesis would not have been possible.

Finally, I thank everyone who supported me along the way, as your encouragement meant a great deal to me.

Abstract

Engineering systems often describe one product with several models that share information, so a change to one model has to be propagated to all other models that depend on it. When many updates arrive concurrently, this propagation can either be done inside each transaction (ACID), which keeps the models consistent but limits throughput, or be moved to the background (BASE), which achieves high throughput but leaves the models temporarily inconsistent without giving the administrator control over how much inconsistency is accepted.

This thesis addresses this trade-off with a hybrid execution mode that decides for each transaction whether its changes are propagated immediately or in the background, so that both kinds of work can run side by side on the same data without conflicting. The decision is made by a mode oracle, and this work presents two of them, one that applies immediate propagation to a fixed share of transactions and one that adapts to the current level of inconsistency in the system. The hybrid mode is implemented in an existing simulator of multi-model systems, which makes it possible to evaluate how well it balances consistency and throughput.

The evaluation shows that the hybrid mode reaches a higher throughput than pure ACID and, unlike pure BASE, keeps the inconsistency close to a configured threshold. Under a static load, however, the adaptive oracle gives no performance gain over the fixed share.

Kurzfassung

In der Systementwicklung wird ein Produkt oft durch mehrere Modelle beschrieben, die Informationen teilen, sodass eine Änderung an einem Modell auf alle anderen Modelle übertragen werden muss, die davon abhängen. Wenn viele Änderungen nebenläufig eintreffen, kann diese Übertragung entweder innerhalb jeder Transaktion erfolgen (ACID), was die Modelle konsistent hält, aber den Durchsatz begrenzt, oder in den Hintergrund verlagert werden (BASE), was einen hohen Durchsatz erreicht, die Modelle jedoch zeitweise inkonsistent lässt, ohne dass der Administrator steuern kann, wie viel Inkonsistenz akzeptiert wird.

Diese Arbeit begegnet diesem Zielkonflikt mit einem hybriden Ausführungsmodus, der für jede Transaktion entscheidet, ob ihre Änderungen sofort oder im Hintergrund übertragen werden, sodass beide Arten von Arbeit ohne Konflikte nebeneinander auf denselben Daten laufen können. Die Entscheidung trifft ein Modus-Orakel, von denen diese Arbeit zwei vorstellt, eines, das einen festen Anteil der Transaktionen sofort überträgt, und eines, das sich an den aktuellen Grad der Inkonsistenz im System anpasst. Der hybride Modus ist in einem bestehenden Simulator für Multi-Modell-Systeme implementiert, wodurch sich bewerten lässt, wie gut er Konsistenz und Durchsatz ausbalanciert.

Die Evaluation zeigt, dass der hybride Modus einen höheren Durchsatz als reines ACID erreicht und, anders als reines BASE, die Inkonsistenz nahe an einem konfigurierten Schwellenwert hält. Unter statischer Last bringt das adaptive Orakel jedoch keinen Leistungsgewinn gegenüber dem festen Anteil.

Contents

Acknowledgments	iii
Abstract	iv
Kurzfassung	v
1. Introduction	1
1.1. Motivation	1
1.2. Formal problem statement	1
1.3. Research questions	2
1.4. Approach and contributions	2
1.5. Structure of the thesis	2
2. Preliminaries	4
2.1. Transactions, ACID, and BASE	4
2.2. The base simulator	4
2.3. Multi-model systems and consistency preservation	5
2.4. Hysteresis band	7
2.5. Notation	7
3. Related Work	8
3.1. Simulation of transactional systems	8
3.2. Deciding an execution mode at runtime	9
3.3. Combining strong and weak consistency	9
3.3.1. Static classification of transactions	10
3.3.2. Adaptive and bounded consistency	10
3.4. Multi-model consistency preservation	11
3.5. Discussion and delimitation	11
4. Approach	12
4.1. Starting point and contributions	12
4.2. Introduction of hybrid mode	12
4.3. Received simulator allows only one mode per run	13
4.4. Implementing the hybrid mode	14
4.5. The shared-resource arbiter	16
4.6. Mode oracles	22
4.7. Safety of the hybrid mode	26

5. Experiments	29
5.1. Experimental setup	29
5.2. X1: Reconciler-processor sweep	31
5.3. X2: Critical dirty fraction threshold sweep	35
5.4. X3: Fixed-share oracle ACID/BASE split sweep	39
5.5. X4: Semantic overlap sweep	43
5.6. X5: Max number of resources per transaction sweep	45
5.7. The effective recovery threshold	47
5.8. Threats to validity	49
6. Conclusions and Future Work	50
6.1. Summary of the work	50
6.2. Lessons learned	50
6.3. Future work	50
A. Use of AI Tools	52
Bibliography	53

1. Introduction

This chapter introduces the problem of this thesis and the approach taken to solve it. Section 1.1 explains why the problem exists. Section 1.2 states the problem. Section 1.3 lists the research questions. Section 1.4 summarizes the approach and the contributions. Section 1.5 gives the structure of the thesis.

1.1. Motivation

Engineering systems often describe one product with several models. Examples are the CAD geometry, a simulation mesh, and a written specification. These models overlap because the same fact about the product appears in more than one of them. Consistency Preservation Rules (CPRs) keep the models in agreement. They take an update made in one model and propagate it to every model that shares the changed resource.

When many updates run at the same time, there are two ways to apply a change. The first way is consistent but slow. The change is accepted only after every affected model is ready to apply it. The more models a change reaches, the longer this takes, so performance drops sharply as the number of models grows [1]. The second way is fast. The change is accepted immediately, and the work of propagating it is moved to the background. This background work is called *reconciliation*. Until it is done, the other models that share the resource still show the old state. They see the change only after some delay.

In transactional systems, these two ways are known as ACID and BASE. Chapter 2 introduces both in detail. This thesis combines them in one system. The goal is to find a point between the two that is faster than the consistent way alone, while the delay in the other models stays within a limit that the system administrator accepts.

1.2. Formal problem statement

Delays in background updates may leave some resources out of date. This thesis measures this inconsistency as the share of resources that are out of date. We call this share the *dirty fraction* d (Section 2.3). Let c be the *critical dirty threshold*, which is the highest dirty fraction that the system administrator accepts. The task is to build a mechanism that keeps the dirty fraction d below c for most of the run. More precisely, the 95th percentile of the dirty fraction over the run, $p_{95}(d)$, has to satisfy

$$p_{95}(d) \leq c + \varepsilon \tag{1.1}$$

where ε is a small tolerance for deviations. The threshold should be set once, and while the system is running, the mechanism has to keep the dirty fraction at this level on its own, so the administrator does not have to adjust any setting.

1.3. Research questions

- RQ1** How can a multi-model transactional system integrate a *hybrid mode* that safely adapts between ACID and BASE semantics at runtime, when its subsystems hold overlapping resources?
- RQ2** How does the hybrid mode compare with pure ACID and pure BASE execution in throughput and inconsistency, and can the inconsistency it permits be configured?
- RQ3** Does choosing the mode from the system's own internal metrics outperform a fixed ACID/BASE split on the same throughput/inconsistency trade-off?

1.4. Approach and contributions

This thesis builds on the simulator of multi-model systems by Zacouris and Acosta [2, 1]. The simulator supports both ACID and BASE semantics, but only one of them per run. This thesis adds the following.

- **A hybrid execution mode.** The mode is chosen for each transaction instead of once for the whole run. Both semantics run side by side on the same resources (Section 4.4).
- **A shared-resource arbiter.** When ACID and BASE run together, they may need the same resource at the same time. The arbiter manages this access so that no resource is corrupted (Section 4.5).
- **Two mode oracles.** An oracle chooses the mode of each transaction, either ACID or BASE. The adaptive oracle decides from the internal state of the system, namely the measured dirty fraction. The fixed-share oracle is the alternative. It uses a fixed split between ACID and BASE that is set at the start (Section 4.6).
- **Metrics for hybrid runs.** Additional logs and metrics that compare performance and describe the internal state of the system (Section 5.1).
- **An empirical evaluation.** Five parameter sweeps with 620 simulation runs in total (Chapter 5).

1.5. Structure of the thesis

Chapter 2 introduces the background needed to understand this work. It covers transactions, ACID and BASE, the base simulator, multi-model systems, and hysteresis. Chapter 3 discusses the literature this thesis builds on. Chapter 4 describes the hybrid mode, the shared-resource

arbiter, and the two mode oracles. Chapter 5 describes the experimental setup, reports the results, and maps them to the research questions. Chapter 6 summarizes the work, records the lessons learned, and lists future work.

2. Preliminaries

This chapter describes the background that the later sections use. Section 2.1 defines a transaction, the ACID guarantees, the locking rule that provides them, and BASE. Section 2.2 describes the basics of the simulator that this thesis extends. Section 2.3 explains the multi-model system it simulates, where a write in one model requires updates in other models. Section 2.4 covers the hysteresis band, which the adaptive oracle uses. Section 2.5 lists the symbols used in the thesis.

2.1. Transactions, ACID, and BASE

Transactions. A transaction is a group of read and write steps on resources that takes effect as one unit. When several transactions run at the same time, the result must be the same as if they had run one after another. This property is called serializability.

ACID. ACID names four guarantees that a transactional system gives. Atomicity means that all steps of a transaction take effect or none of them do. Consistency means that a committed transaction leaves the data in a valid state. Isolation means that concurrent transactions do not see each other's unfinished work. Durability means that a committed result is not lost afterwards. Isolation is the most expensive of the four, and it is provided by locking.

Locking. A transaction declares the resources it will work with and has to acquire a lock on each of them before it can start. No other transaction can lock these resources until the locks are released after the last step. Acquiring the whole set in advance is called *preclaiming*, and the scheme is known as conservative two-phase locking. It rules out deadlock, because a transaction never waits while it holds a lock [3].

BASE. BASE gives up immediate consistency to gain availability and throughput [4, 5]. A transaction commits as soon as its own work is done. It does not wait for the work it causes in the rest of the system. That work runs later in the background, and the system becomes consistent again over time.

2.2. The base simulator

This thesis extends the simulator of Zacouris and Acosta [2, 1]. This section covers only its basics.

The execution engine. Time advances in fixed steps of one simulated second. A new transaction waits in a pending queue until it is admitted. It then passes through a lock phase, an IO phase, and a CPU phase on pools of IO and CPU processors, and finally, it commits or aborts. ACID transactions use conservative two-phase locking (Section 2.1). BASE transactions skip the lock phase, and conflicting writes are resolved by a last-write-wins rule. The clock is virtual, so the results do not depend on the speed of the machine.

Simulator parameters. A run is configured by the number of resources, models, and locks, the number of IO and CPU processors, and the execution mode, ACID or BASE, which holds for the whole run. It is also configured by the maximum number of resources and of models that one transaction may touch. The multi-model parameters are introduced in Section 2.3. Table 5.1 gives the values used in this thesis.

Workload. The simulator creates its own transactions. Each transaction draws at random how many resources and which models it touches, which is reproducible with a seed. This thesis uses the EQUILIBRIUM profile, which keeps the number of in-flight transactions constant. Whenever one transaction finishes, a new one is created. A second profile, TERMINAL, lets the load rise and fall, but it was still under development and is not used.

2.3. Multi-model systems and consistency preservation

Models and semantic overlap. A model is a set of resources that describes one view of a product [6]. A multi-model system is a finite set of resources $\mathcal{X}$ with $|\mathcal{X}| = N$, divided between the models $M_1, \dots, M_m$. Two models have *semantic overlap* when they contain resources that represent the same fact [1].

Consistency preservation rules. A consistency preservation rule (CPR) states that a write to one resource requires updates to the resources that overlap with it. In the notation of this thesis, a rule maps a write on a resource x to the set $\text{cpr}(x) \subseteq \mathcal{X}$ of resources that must be updated as a consequence [1]. An update made by a rule can trigger further rules, up to a maximum depth. One write can therefore reach resources in several models. Figures 2.1 and 2.2 show such a chain of rules across five models.

Bookkeepers. A transaction that arrives from the workload is called a *root* transaction. When it writes a resource x , the resources in $\text{cpr}(x)$ must also be updated. These updates are made by separate transactions, called *bookkeepers*. A bookkeeper may itself trigger further rules, and then it creates bookkeepers of its own. Every bookkeeper runs in the same execution mode as the transaction that created it. *Rule expansion* is the step that creates the bookkeepers of a transaction. The following paragraphs describe how ACID and BASE execute the bookkeepers.

Transaction groups. A root transaction and all the bookkeepers that follow from it form a *transaction group* [1]. Under ACID, the group commits as one unit, and if one member is blocked or aborted, so is the whole group. The bookkeepers are not known when the root is

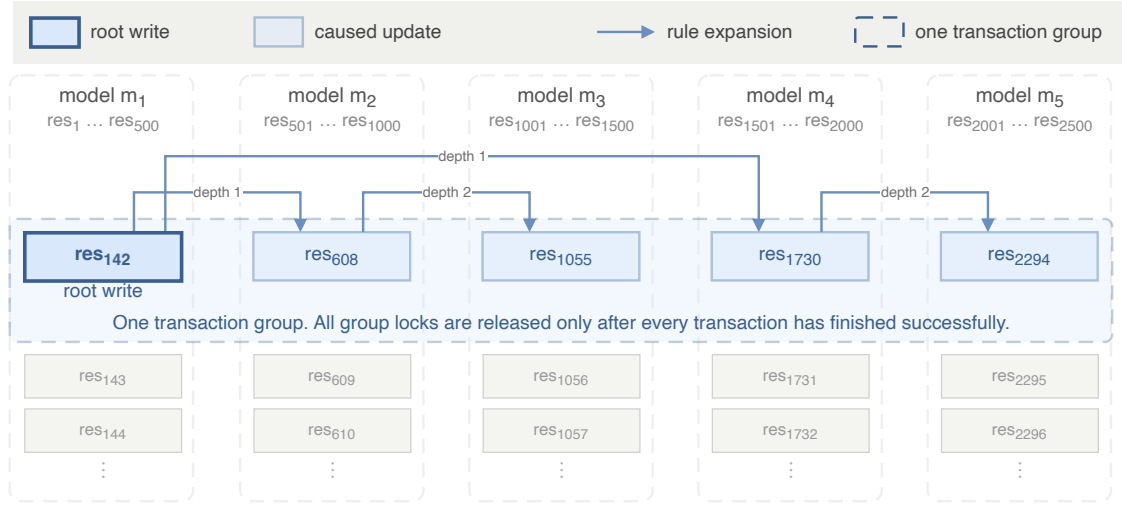


Figure 2.1.: Rule expansion under ACID.

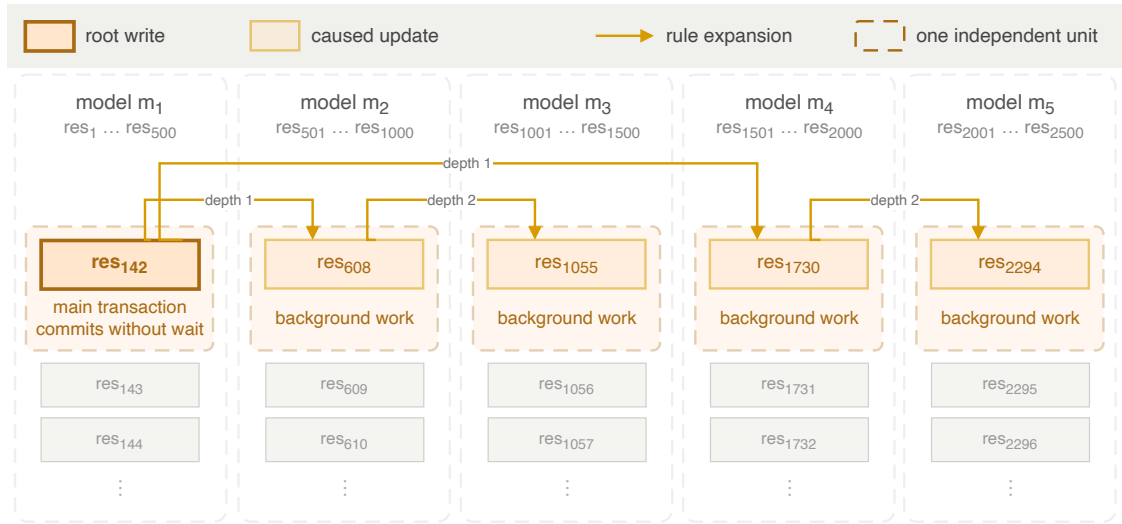


Figure 2.2.: Rule expansion under BASE.

admitted, so the root preclaims only its own locks. The bookkeepers add their locks later, and all locks are released when the group commits (Figure 2.1). A group can therefore wait while it holds locks, and two groups can wait on each other, which can result in deadlock.

Cautious waiting. The base simulator prevents such a deadlock with cautious waiting [7, 1]. A transaction may queue for a lock only if the group that holds it is not blocked itself. Otherwise, its group is aborted, releases all of its locks, and is retried later. A pure ACID run, therefore, cannot deadlock, but it loses aborted work (Section 4.3).

Dirty resources and reconciliation. Under BASE, a root transaction commits without waiting for its bookkeepers. Until a bookkeeper has committed, the resource it updates holds an old value and is called *dirty*. *Reconciliation* runs the pending bookkeepers in the background on dedicated reconciler processors, taken from the foreground IO and CPU processors (Figure 2.2). Under ACID, the number of reconciler processors is ignored.

Inconsistency. The dirty set $D_k \subseteq \mathcal{R}$ holds the dirty resources at reading k , and $d_k = |D_k|/N$ is the *dirty fraction*. A run is summarized by its 95th percentile $p_{95}(d)$, which is the value bounded in Equation 1.1. Under pure ACID, d_k is always zero.

2.4. Hysteresis band

A policy that switches between two states with a single threshold switches at every crossing when the measured value moves around it. It pays the cost of each switch without gaining from it. A *hysteresis band* uses two thresholds instead, one for entering a state and a lower one for leaving it. The next switch happens only after the value has crossed the whole band. Section 4.6 applies this idea to the adaptive oracle.

2.5. Notation

Table 2.1 lists the symbols used in the thesis. The index k counts readings of the dirty fraction. The reported percentiles use a reading at the end of every simulated step.

Table 2.1.: Notation.

Symbol	Meaning	Definition	Implementation
N	Total resources	—	<code>totalResources</code>
σ	Semantic overlap degree	—	<code>semanticOverlap</code>
L	Max resources per txn	—	<code>maxTransactionResources</code>
R	Reconciler processors	—	<code>numberOfReconcilerProcessors</code>
D_k	Dirty resource set	—	<code>DirtySetManager.size()</code>
d_k	Dirty fraction	$ D_k /N$	<code>dirtyFraction</code>
$p_{95}(d)$	Dirty-fraction tail	95th pct.	<code>dirty_p95_pct</code>
c	Critical dirty threshold	—	<code>criticalDirtyFraction</code>
r	Recovery threshold	—	<code>recoveryDirtyFraction</code>
p_{acid}	Configured ACID share	—	<code>fixedAcidShare</code>
ϕ_{time}	ACID time share	—	<code>acid_fraction_time</code>
ϕ_{comp}	ACID completion share	—	<code>acid_fraction_completed</code>

3. Related Work

This chapter describes how the thesis integrates into existing literature. Section 3.1 covers transaction simulators and the base simulator that this thesis extends. Section 3.2 explains systems that already choose an execution mode per transaction at runtime. Section 3.3 describes work that combines strong and weak consistency in one system. Section 3.4 focuses on the multi-model framework. Section 3.5 summarizes what was built upon and what was added with this thesis.

3.1. Simulation of transactional systems

It is hard to study concurrency control in real-world systems. As a result of that, we will work with a simulator that allows us to compare different policies under the same synthetic workload. This section first covers two single-model studies and then the two papers of the base simulator.

Single-model simulators. Ries and Stonebraker [3] simulate the load of a database to study how the size of locks affects performance. Dandamudi and Au [8] repeat this study for a database that runs on several processors. Both simulators keep a fixed number of transactions in-flight. Each transaction moves through a specific pipeline. It starts with a pending queue, an IO queue, and a CPU queue, and it waits in a blocked queue when a lock it needs is taken.

The base simulator, and therefore this thesis, takes this structure of queues and the fixed number of transactions. The base simulator [1] is also validated against the results of Dandamudi and Au [8]. Both studies consider a single model with ACID execution only.

The multi-model simulator M2TS. Zacouris and Acosta [2] carry this method over to multi-model systems. In their simulator, an update to one model can trigger updates in others. The simulator supports ACID only.

Its main result is that throughput drops towards zero as the number of models increases, and that lock acquisition is the bottleneck.

ACID and BASE in the same simulator. The follow-up study [1] adds the BASE semantic and is the direct base of this thesis. Under BASE, a transaction takes no locks, and the triggered updates run later in the background on dedicated reconciliation processors. The study also measures inconsistency as the share of resources that wait for such an update.

Its results are the starting point of this thesis. Under ACID, throughput falls sharply as the overlap of the models increases. Under BASE, throughput is almost unaffected, but the system carries a temporary inconsistency. The amount of inconsistency is controlled by the number of reconciliation processors.

In the base simulator, ACID and BASE are two separate configurations, and one run uses only one of them. This thesis lets both modes run in the same run, chooses the mode per transaction at admission, and makes the inconsistency a configurable parameter.

3.2. Deciding an execution mode at runtime

The work in this section is the closest to the design of this thesis. These systems tag each transaction with one of several execution modes at runtime. The most important of them is Hybrid Transactional Replication, because several parts of the approach in Chapter 4 follow its approach.

Hybrid Transactional Replication. Kobus et al. [9] study one service whose data is fully replicated on several machines. Each transaction runs in one of two modes. In *state machine* mode, the request is sent to all replicas, and every replica executes it in the same order. In *deferred update* mode, only the replica that received the request executes it, and at commit, its updates are sent to all other replicas, which check them for conflicts and apply them. A component called the *oracle* chooses the mode for each transaction. Their learned oracle, HybML, reaches up to 50% higher throughput than using only one of the two modes.

This thesis takes three ideas from HTR. The mode is chosen per transaction, both modes are active in the same system, and the choice is made by an oracle behind a small interface (Section 4.4).

The main difference lies in the configuration. HTR replicates one data set, and a replica never creates background work. This thesis focuses on a multi-model system, where independent models share some of the resources, receive their own requests, and trigger updates in each other.

The second difference is what the modes guarantee. Both HTR modes are strongly consistent, while BASE in this thesis leaves dirty resources for background reconciliation. This creates the shared state that the arbiter has to manage (Section 4.5).

Finally, HybML maximizes throughput, while the adaptive oracle in this thesis focuses on a configurable dirty resource fraction threshold.

PolyCert and MorphR. PolyCert [10] makes the same kind of choice in replicated in-memory data grids, where a learned oracle picks one of three certification protocols for each transaction. MorphR [11] switches the replication protocol of the whole system instead of a single transaction. As in HTR, all their modes keep the data strongly consistent, and the oracles optimize only throughput and latency.

3.3. Combining strong and weak consistency

The work in this section lets strong and weak consistency exist in the same system, as ACID and BASE do in this thesis. Section 3.3.1 covers Salt and RedBlue consistency. They run strong and weak work side by side, but the consistency type has to be set by a developer before the

system runs. Section 3.3.2 covers Consistency Rationing, OptCon, epsilon-serializability, and TACT. They change the consistency level at runtime or put an inconsistency under threshold, but none of them decides whether a whole transaction with its triggered updates runs as ACID or BASE.

3.3.1. Static classification of transactions

Salt. Xie et al. [12] make ACID and BASE transactions run in the same distributed database. A developer rewrites only the few transactions that limit performance as BASE transactions, and all other transactions stay ACID. The rule that keeps this combination safe is *Salt Isolation*. To an ACID transaction, a BASE transaction looks like just another ACID transaction, so ACID never sees unfinished BASE work. BASE transactions, however, may see each other's work in progress, so their locks collide less often.

RedBlue consistency. Li et al. [13] make a similar split for geo-replicated systems, but per operation. A *blue* operation runs first at the node that received it, without waiting for others, and is sent to the other nodes later. Different nodes may therefore apply blue operations in different orders. A *red* operation is placed in one order that all nodes follow, so it needs coordination. An operation may be blue only if the order does not change the final state, and if it does not corrupt the system state.

Both papers show that strong and weak consistency can safely run in one system. Salt is also the closest precedent for the arbiter of this thesis, because its ACID transactions wait for BASE work to finish, while BASE transactions may see each other's unfinished work. The claim table of Section 4.5 follows the same idea. What these papers do not do is decide the mode at runtime. In both systems, the split is made by hand and is fixed in the source code before the system runs. They also do not measure or bound how much inconsistency exists.

3.3.2. Adaptive and bounded consistency

Consistency Rationing. Kraska et al. [14] attach consistency guarantees to data instead of to transactions. Some data is always serializable, some only has weak consistency, and the rest switches between the two at runtime, for each record. In a web shop, for example, account balances are always serializable, user preferences are weakly consistent, and the stock of a product becomes serializable only when few items are left. A policy makes the switch, for example, when the estimated probability of a conflict becomes too high. The goal is to lower the total cost of transactions and of inconsistencies.

OptCon. Sidhanta et al. [15] choose a consistency level for every read and write in a quorum-based store such as Cassandra. The user states a latency limit and a staleness limit, and a trained model predicts which consistency level meets both.

Epsilon-serializability and TACT. Pu and Leff [16] and Wu et al. [17] let a read-only transaction see a limited amount of inconsistency, while update transactions stay serializable. Yu and Vahdat [18] bound how far the copies of replicated data may drift apart, with limits

that the application sets for each unit of data. Both approaches treat inconsistency as a number with a limit. The critical dirty threshold of this thesis is a limit of this kind.

All of this work changes or bounds consistency based on what the system observes, which is the idea behind the adaptive oracle and critical dirty threshold. The difference is what is decided and where the limit is set. These papers work per record, per operation, or per unit of data, but none of them decides whether a whole transaction and the repairs it triggers run as ACID or BASE. Their limits are also set for a single transaction, as in epsilon-serializability, or for single units of data, and none keeps one inconsistency level for the whole system, like the dirty fraction in this thesis.

3.4. Multi-model consistency preservation

Vitruvius. Klare et al. [6] provide the setting of this thesis. Vitruvius keeps the overlapping models of a system separate and connects them into a *virtual single underlying model*. A change in one model is propagated to the other models by consistency preservation rules. This is the mechanism that creates the workload studied here, and the base simulator is calibrated with measurements from two systems built with Vitruvius [1].

Vitruvius applies changes sequentially. After each change, the rules run until all models are consistent again, and only then is the next change started. This gives a consistent state after every change without any concurrency control.

3.5. Discussion and delimitation

Except for the two papers of the base simulator, all work focuses on data under a single schema. There, inconsistency means, for example, replicas that differ, unfinished work that other transactions may see, or a query result that is off by a limited amount. The inconsistency that updates create between overlapping models is not addressed. Vitruvius addresses these models, but it has no transactions and no deferred repairs. This thesis works in the space between the two. It chooses ACID or BASE per transaction at runtime in a multi-model system, and keeps the resulting inconsistency close to a configured inconsistency threshold.

4. Approach

This chapter describes how the received simulator was extended so that it supports the hybrid mode. Section 4.1 lists which parts are inherited and which are new. Section 4.2 explains why a hybrid mode is needed. Section 4.3 shows how the received simulator runs only one mode per run. Section 4.4 adds the hybrid mode on top of it. Section 4.5 covers the shared-resource arbiter, which keeps ACID and BASE from breaking each other’s work. Section 4.6 describes the two oracles that choose the mode of each transaction. Section 4.7 defines what *safe* means in this work and argues that the hybrid mode meets this definition.

4.1. Starting point and contributions

The base simulator described in Section 2.2 is existing work [2, 1]. Two tables make clear which parts are inherited and which are new. Table 4.1 lists the components that this thesis builds on. Each of them is used as it was received.

Table 4.1.: Taken from the base simulator.

Execution engine	Fixed time step, IO and CPU processor pools
ACID runtime	Lock table, conservative 2PL with cautious waiting
BASE runtime	Dirty set, its (model, resource) key, fan-out collapse, and the reconciler pool that clears it
Rule expansion	Bookkeeper transactions and multi-model CPR
Workload generator	Seeded transactions under the EQUILIBRIUM profile

4.2. Introduction of hybrid mode

The purpose of the hybrid mode is to combine the ACID and BASE semantics in the same run. The share of each mode then becomes a way to control metrics such as throughput and inconsistency. ACID is the slow but consistent approach. Its dirty fraction d_k is always zero, but its throughput drops sharply as the number of resources and models grows [1]. BASE hands the work that has to be propagated to other models over to the reconciler pool, which clears the updates eventually. However, BASE gives no control over the inconsistency of the system, and the dirty set can grow faster than the reconcilers are able to clear it.

Table 4.2 lists the components of the hybrid mode that were introduced in this thesis, with a reference to the section where each of them is described in more detail.

Table 4.2.: Added in this thesis.

HYBRID execution mode (Section 4.4)	The mode becomes a per-transaction tag instead of a run-wide configuration
Shared-resource arbiter (Section 4.5)	Lookup that connects the ACID and BASE bookkeeping on shared resources, so that neither mode overwrites the work of the other
Claim table (Section 4.5)	Records the locks used by BASE parent transactions
Fixed-share oracle (Section 4.6)	Tags ACID with a configured probability p_{acid}
Adaptive oracle (Section 4.6)	Switches to ACID when the dirty fraction reaches the threshold, and back to BASE once the inconsistency has recovered
Hybrid metrics	Mode switches, ACID share, blocking, and ACID visit duration

4.3. Received simulator allows only one mode per run

The simulator can run ACID or BASE, but never both. A run binds its configured concurrency model through the *runtime assembler*, and there is one assembler per mode. It builds every mode-dependent collaborator at the start of the simulation, so all transactions run in the one mode that was assembled at the beginning. Figure 4.1 shows how the mode-specific components are wired. While explaining how the two modes are started, this section also follows the path that one ACID or one BASE transaction takes, because the hybrid mode reuses that path.

At the start of the simulation, the runtime assembler (Step 1 in Figure 4.1) creates either ACID or BASE wiring. Only one path exists in the simulator at a time. For example, when the simulator is configured in ACID mode, none of the BASE components are built, and vice versa. Only the parts that do not depend on the mode, such as the execution engine and the first part of admission, are the same in both wirings.

Admission (Step 2) has two parts, and the rest of this chapter keeps them apart. The admission controller takes the next transaction from the pending queue. The second part is the *mode admission step*, which admits the transaction only while the number of transactions in flight is below the capacity limit, and it differs between the modes.

BASE has no locking step and admits the transaction straight into execution. ACID first acquires the whole lock set of a parent transaction at once, or waits while holding none of it. A transaction counts as *admitted* only once it has passed the mode admission step. A new transaction that arrives from the workload is not admitted yet. It only joins the pending queue. The concurrency strategy and the transaction executor (Steps 3 and 4) then drive the phases LOCK, IO, CPU, and COMPLETE, and BASE skips LOCK.

In BASE mode, the processor pool is split into two sets. The first is the foreground processors, which handle the initial parent transactions. The second is the reconciler pool, which propagates the changes to the other models. The simulator allows both the number of foreground IO and CPU processors and the number of reconciler processors to be configured. In BASE, the number of reconciler processors is subtracted from the pool of foreground processors. In ACID, the reconciler processor count is ignored. The hybrid mode described

below keeps the BASE rule, so in a HYBRID run, the ACID transactions also use the smaller foreground pool. After a parent transaction is processed, rule expansion (Step 5) decides what happens next. ACID parks the parent in the waiting room (WAITING ROOM in Figure 4.4), where it waits for its group, and spawns its bookkeepers, which have to go through the same process of acquiring locks. BASE commits the parent immediately and then starts its bookkeepers independently, without waiting for them.

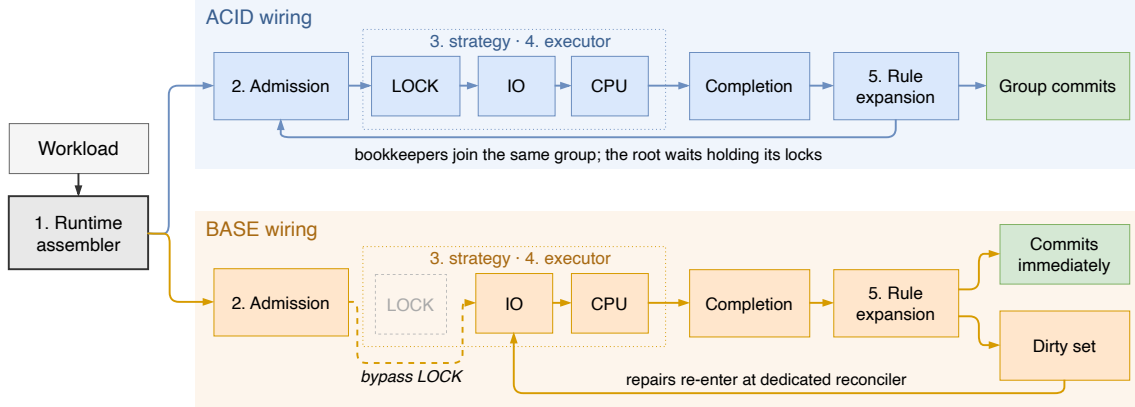


Figure 4.1.: The received pipeline.

One property of the ACID mode is important. The unit of locking is the group and not the transaction, because the root preclaims its own set at admission while its bookkeepers extend that set afterwards. A group can therefore be blocked while it already holds locks, and two groups can wait on each other. The received simulator prevents this with cautious waiting (Section 2.3), which uses two abort rules. First, a transaction that asks for a lock held by a blocked group is aborted instead of being queued. Second, when a group becomes blocked, the transactions already queued on its locks are aborted.

The components of Steps 2 to 5 each exist in an ACID form and a BASE form, and the assembler of Step 1 picks one form at the start. A run, therefore, inherits one execution mode and keeps it to the end.

The idea of the hybrid mode is to keep both modes and to wrap each of these components in a dispatcher that holds one collaborator of each kind.

4.4. Implementing the hybrid mode

A third execution mode, HYBRID, is added next to ACID and BASE. It is not a third set of semantics, but a configuration in which both received sets are live at the same time. What the hybrid mode adds is the choice of which mode a transaction runs in.

Dispatchers wrap both paths. A third runtime assembler builds both mode-specific sets in full and wraps each pair. Every component of Section 4.3 is then filled by a dispatcher that holds one ACID collaborator and one BASE collaborator, and forwards each call to the

one that the transaction names. Both ACID and BASE paths are left unchanged. The two admission controllers share the scheduler, the processors, the I/O model, the queue manager, and the capacity limit, and the two bookkeeper coordinators share the dirty set and the lock table. The dispatchers and the arbitration checks are built by the hybrid assembler only, so a pure ACID or pure BASE run still executes as in the received simulator.

Introduction of tag and mode oracle. Since both modes are wired into the simulator at the same time, the decision which path to take has to move out of the assembler. Our approach introduces a tag for each transaction. Only two tags exist: $\{ACID, BASE\}$. Each transaction receives its tag once, and the tag is propagated to the bookkeepers of that transaction. Choosing an execution mode per transaction at runtime is not new in itself. Hybrid Transactional Replication [9] and PolyCert [10] already do it, but there, all modes give the same consistency, so the choice is never about controlling the inconsistency level (Section 3.2).

The tag is chosen by a component called the *mode oracle*. It is implemented as a Java interface with a single method, `assignMode()`. This thesis introduces two mode oracles, which are described in Section 4.6. The oracle is called as soon as the admission controller takes an untagged parent transaction from the pending queue. The mode oracle returns the mode in which the transaction should run. The dispatcher then hands the transaction to the admission controller of that mode, which runs the *mode admission step*, either ACID or BASE. A tagged transaction that finds no free capacity keeps its tag and waits for capacity like any other.

Tags are immutable. They cannot be withdrawn or changed. Once a transaction has received its tag, it stays in that mode for its whole life, and so do all of its bookkeepers. The same holds when a transaction is aborted. It loses all its progress, but not its tag.

Sharing one resource pool and the limits of the naive design. One set of resources is created at the beginning and is never divided again. At the baseline, that is 5000 resources, divided equally between the 10 models. An ACID-tagged transaction may lock any of the 5000 resources, and a BASE-tagged transaction may dirty any of the same 5000.

Simply wrapping both modes is a naive design here. It separates the path a transaction takes, but the resources that the transaction needs are still shared. The problem is that the lock table used by ACID and the dirty set used by BASE do not communicate. As Figure 4.2 shows, both modes ignore the progress of the other when they work on the same resource. ACID locks are visible only to other ACID transactions, and BASE dirty marks only to other BASE transactions. This would break the isolation that ACID promises, because a locked resource could be overwritten while it is locked.

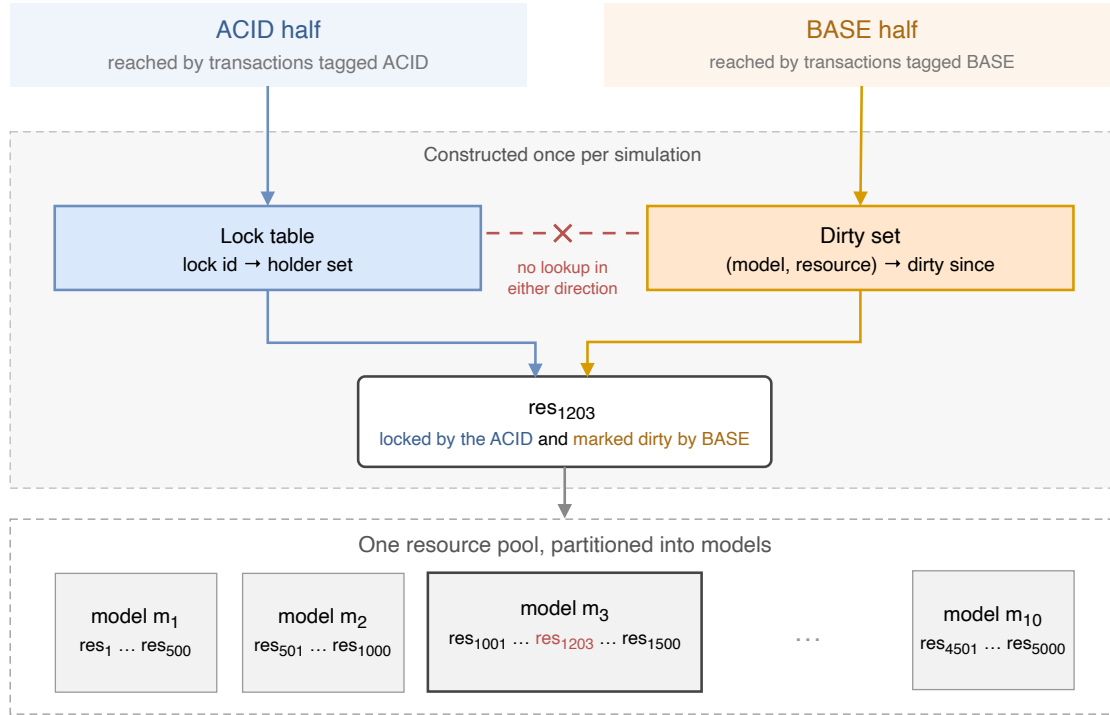


Figure 4.2.: Resource management in the hybrid mode.

A resource that is ACID-locked and BASE-dirty at the same time is in an undefined state, because each half relies on an assumption that the other half breaks. Sharing the pool is therefore only half of what a hybrid run needs. The other half is a shared-resource arbiter. It connects the bookkeeping of the two pure modes and defines what a mode does when a resource it needs is in use by the other mode. The next section addresses this problem.

4.5. The shared-resource arbiter

Section 4.4 left the hybrid mode with one open problem. Both halves write to the same resources and ignore the work of the other half. The lock table is asked only whether a lock is free. The dirty set is asked only whether a repair is in progress. Nothing in that scenario stops a resource from being locked and dirty at the same time.

This section describes the shared-resource arbiter, the mechanism introduced to solve this isolation problem between the ACID and BASE bookkeeping.

Arbiter implementation. The arbiter is not a single class that holds references to all resources or locks. It reuses two existing records, the lock table and the dirty set, and adds a third one, the claim table. Figure 4.3 shows how the three records are connected. Every lookup starts from the same place, the name of a resource in the shared pool (column 1). An ACID transaction, a BASE parent, and a BASE repair all start from that name when they want to

use resource res_k . The name is then mapped to a key (column 2). The lock table and the claim table both use the lock id, which the granularity mapping computes from the resource. A run may be configured with fewer locks than resources, so one lock id can cover several resources. The dirty set uses the dirty key, the pair of the owning model and the resource, so it always has one entry per resource. Under its key, each record keeps one entry (column 3). The lock table lists the ACID transactions holding the lock, the claim table lists the BASE parents writing under it, and the dirty set holds the repair that is still outstanding on the resource. Because both keys are computed from the same resource name, any path can look up any record without extra bookkeeping.

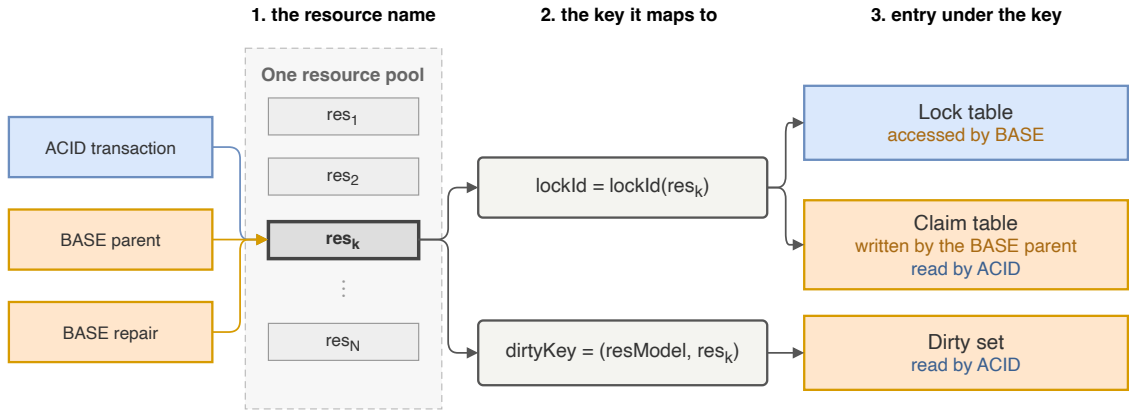


Figure 4.3.: The shared lookup the arbiter is built on.

The colors in Figure 4.3 show who writes and who reads each record. The lock table is written and read by ACID as before, and is now also read by both BASE paths. The claim table is written by BASE parents and the dirty set by BASE repairs, and both are read by ACID. Each record is therefore still written by one side only. What the arbiter adds are reads across the two modes.

These reads are placed in the mode admission step (Step 2 in Figure 4.1), which is extended to also check the state of the opposite mode. For example, when an ACID transaction goes through its mode admission step, the arbiter checks whether any resource it needs is dirty or claimed, by looking it up in the dirty set and in the claim table. A transaction that fails this check is not admitted. It has already left the pending queue, but it goes to the waiter registry or to the deferred parent queue instead of into execution. There are two exceptions. An ACID bookkeeper is tested in the middle of execution, when it asks for the locks it adds to its group. A BASE repair is tested later, at rule expansion (Step 5), because that is where it is created. The idea is that every path checks a resource for work of the other mode before it uses it.

The components introduced by the arbiter, both to resolve the lookup and to handle a mode that finds a resource in use by the other mode, are the following.

- **Claim table.** One entry per lock under which an admitted BASE parent is currently writing. A parent enters its claims when it is admitted and removes them when it completes.

- **Waiter registry.** A list of observers. An ACID transaction that needs a dirty or claimed resource registers itself under that resource and stops. When the repair finishes or the claim is dropped, the registry notifies every transaction registered under it, and each one retries.
- **Deferred parent queue.** The BASE parents that met an ACID lock at admission. BASE transactions are not treated as observers and have a polling retry.
- **Deferred repair queue.** The BASE repairs whose target was ACID-locked when the rule that created them fired.

Only the claim table is a new record. The other three components, *Waiter registry*, *Deferred parent queue*, and *Deferred repair queue*, hold the transactions that found a resource in use by the other mode until that resource is free again.

A large simplification is important to mention. The resources are split between models, so in a real system, a lookup takes time because each model that holds the resource has to be reached. These network calls may be slow and can sometimes fail. The simulator sees all resources and models as one entity, so a lookup has no latency at all and always succeeds. Modeling the real system more closely would add a lot of overhead. The simulator, therefore, implements the mechanism in an ideal setting, and it would be less performant in an actual multi-model system.

The invariant. The main goal of the hybrid implementation, and of the arbiter itself, is to enforce one rule. A resource is never held by ACID and written by BASE at the same time. BASE writes a resource in two ways. A BASE parent claims it, and a BASE repair marks it dirty. This keeps the guarantees of ACID transactions intact and addresses the idea of safe adaptation between ACID and BASE posed by RQ1. Because a resource always has one clearly defined state, no work is lost or overwritten. Section 4.7 covers this in more detail.

Shared arbiter lookup position. To enforce the invariant, the pipelines of both ACID and BASE are extended with a lookup for work of the opposite mode. An ACID root is tested at admission, before it takes any lock. An ACID bookkeeper is tested before it takes the locks it adds to its group. A BASE parent is tested at admission, before it claims anything. A BASE repair is tested at rule expansion, before its target is marked dirty.

ACID extended admission step. Figure 4.4 shows how the initial ACID path was extended to look up BASE work. The diagram is adapted from the queueing diagram of Zacouris and Acosta [1]. This thesis extends it with the check against the claim table and the dirty set, and with the waiter registry. An ACID transaction is tested against the claim table and the dirty set together, and both answers have the same effect. If any resource it needs is dirty or claimed, the transaction is paused immediately and added as a waiter to the waiter registry. A root parked this way has not acquired any locks. It stays in the PENDING state, but it is removed from the pending queue. Figure 4.4 shows that it enters the waiter registry, labeled DIRTY WAIT REGISTRY, rather than the lock queue, so it holds nothing while it waits.

An ACID bookkeeper runs the same test in the middle of execution, when it asks for the locks it adds to its group. If the test fails, the whole group is blocked, and its members are

registered as waiters. The group keeps the locks it already holds, but it takes no new one. Section 4.7 explains why this still cannot lead to a deadlock.

Two events make a resource available again. A repair that finishes clears the dirty entry, and a BASE parent that completes drops its claims. A waiter on a dirty resource is registered under the resource, and a waiter on a claim under the lock id, because claims are kept per lock. All waiters registered under the cleared key are then notified and retried. A root parked at admission returns to the pending queue for a fresh attempt, and a bookkeeper parked in mid-execution goes through the existing retry path.

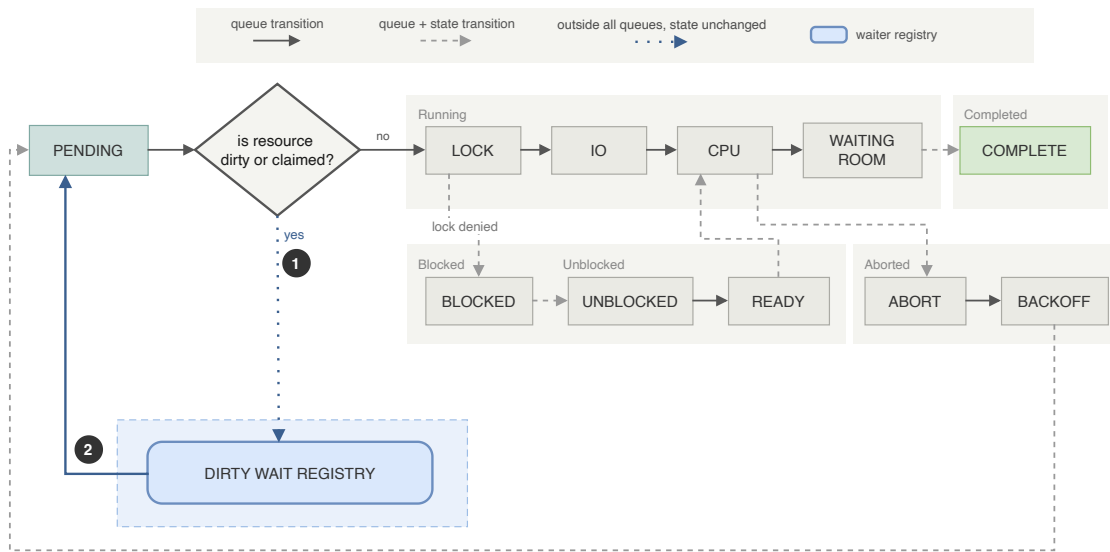


Figure 4.4.: ACID meeting a resource it may not touch.

- ① A resource is dirty or claimed. The root becomes a waiter and holds no lock.
- ② The resource is free again. The waiters are notified and return to the pending queue.

BASE extended admission step. The two BASE paths are tested separately, because they reach a resource at different times. Figure 4.5 shows the parent path, and Figure 4.6 shows the repair path. The parent is the foreground work and is tested at mode admission. Every resource it needs is checked before it claims any of them. If one of them is ACID-locked, the parent claims nothing and goes on the deferred parent queue. Inside this queue, a BASE parent does not wait for a specific ACID transaction. It is retried on every drain, so if the resource is still locked, even by a lock acquired in the meantime, the parent is deferred again rather than overrunning the lock. Once all its resources are free, the parent returns to the pending queue and is admitted like any other. When it completes, it drops its claims and notifies the ACID waiters registered under them.

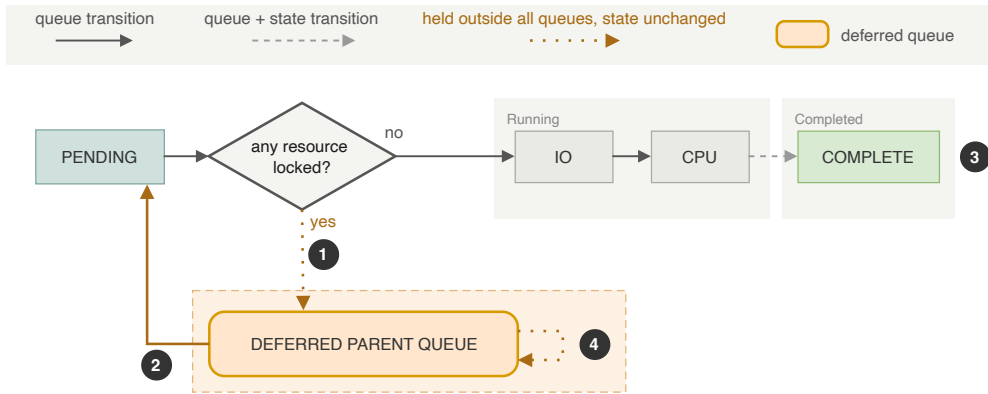


Figure 4.5.: A BASE parent meeting an ACID lock at admission.

- ① A resource is locked. The parent claims nothing and is deferred.
- ② All resources are free. The parent returns to the pending queue.
- ③ On completion, the claims are dropped, and the ACID waiters are notified.
- ④ A resource is still locked. The parent stays deferred.

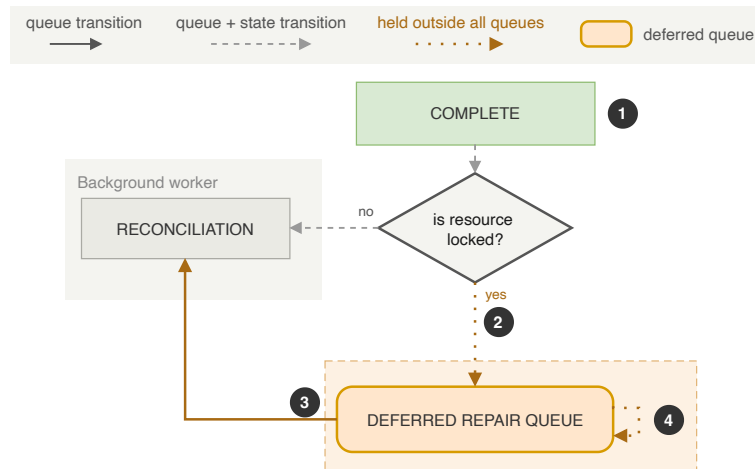


Figure 4.6.: A BASE repair meeting an ACID lock at rule expansion.

- ① The parent commits, and its rule creates a repair.
- ② The target is locked. The repair is deferred, and the target is not marked dirty.
- ③ The lock is gone. The target is marked dirty, and the repair is reconciled.
- ④ The target is still locked. The repair stays deferred.

The repair is tested just before it would mark its target dirty. If the target is ACID-locked, the repair is not created at all. It goes on the deferred repair queue, the target is not marked dirty, and the transaction that triggered it commits. Deferred does not mean waiting. The repair is retried systematically, but there is no guarantee of when it can actually proceed. This avoids a dependency of a bookkeeper on an ACID transaction, which could otherwise lead to a deadlock, and it breaks no promise of eventual consistency. When the repair is finally created, it is created in full and processed.

This has a side effect on what is measured. The parent has already committed, so the target is in fact out of date, but it is not in the dirty set until its repair is created. Neither the oracle nor the reported dirty fraction counts it during that time. The measured dirty fraction can therefore be lower than the real inconsistency. The gap is largest in ACID mode, when many resources are locked.

The claim table is read only by ACID. A BASE parent never checks it, and a dirty resource does not stop it either, so BASE work never waits for other BASE work on the same resource. This follows the idea of Salt Isolation [12], where BASE transactions may see each other's unfinished work, while ACID transactions are kept apart from it (Section 3.3.1).

Both deferred queues are polled rather than woken. Nothing signals an entry when its lock disappears. Parents are retried once per admission step, before anything is admitted. Repairs are retried after every batch of repairs, after every ACID group completion, and once per simulated tick. The pass per tick is the one that matters because a lock is also released by a group that aborts, and an abort is not a completion. Whatever released the lock, the entry is retried within one tick.

Hybrid mode limits the parent BASE transactions. Making a BASE parent wait is where BASE loses performance, and this cannot be avoided. BASE is meant to be the fast mode, which processes the transaction on its own model almost immediately and resolves the consequences in the background. In the hybrid mode, a BASE parent can be held back at admission, so it loses part of that speed.

A BASE parent waits only at admission, only for as long as an ACID lock is held on one of its resources, and only while holding nothing itself. It never blocks after it has been admitted, it never waits on another BASE transaction, and it never waits for reconciliation. It therefore stays much faster than an ACID transaction, but slower than in a pure BASE run.

Where the arbiter is deliberately over-cautious. Two imprecisions are built into the simulator by design.

- **C1. Lock buckets hide free resources.** The two BASE checks and the ACID claim check read the lock of a resource and not the resource itself. Where a run is configured with fewer locks than resources, one lock covers a bucket of them, so BASE work is set aside whenever any resource in that bucket is locked. Transactions, therefore, wait in cases where they could have proceeded. At the baseline, every resource has its own lock (Table 5.1), so C1 has no effect in the experiments of this thesis.

- **C2. Starvation of deferred work.** A deferred entry whose target stays locked can stay on its queue indefinitely. No entry is forgotten, but there is no aging rule that eventually gives a long-deferred entry priority over the ACID transaction keeping its resource busy, so under a resource that is hot for a long time, that work can starve. This is accepted rather than solved. Eventual consistency places no bound on when a repair runs.

4.6. Mode oracles

Section 4.4 gave a brief introduction to what an oracle is. This section describes the two oracles that were built in this thesis. Both use the full hybrid mechanism underneath, with the same interface, arbiter, tagging rules, and dispatchers. They differ only in how the tag is chosen. The `FixedShareModeOracle` is static. It is configured once at the start of the simulation and reads no information about the system. The `AdaptiveModeOracle` is dynamic. It reads the inconsistency level d_k of the system when a transaction is admitted and uses it to choose the tag.

The fixed-share oracle in detail. The fixed-share oracle returns ACID with a configured probability p_{acid} and BASE with probability $p_{\text{base}} = 1 - p_{\text{acid}}$. The draw comes from a seeded generator, so that a run can be reproduced. The tag of each transaction is random, but the expected share of ACID transactions is fixed. Over a long run, the measured share is close to p_{acid} .

It was the first hybrid design, for a practical reason. The value p_{acid} is set at the start of the simulation, so the oracle is easy to configure, and it served as a proof of concept for the hybrid mode. It is also the baseline for RQ3. It is a fixed controller that uses no feedback, and the evaluation checks whether the adaptive oracle can perform better than it.

One design property of this oracle sets how it can be compared with the adaptive one. What p_{acid} fixes is the share of *transactions* tagged to be in ACID mode. It fixes nothing about the share of *time* the system spends in ACID, because the mode of a transaction decides how long that transaction occupies the system.

In short, this fixed-share oracle generates a controlled split of ACID and BASE tags assigned to pending incoming transactions.

The adaptive oracle in detail. The adaptive oracle reads the live dirty fraction d_k at the instant of the decision and switches between two modes. It always starts in BASE mode, because throughput should be as high as possible. As soon as the inconsistency level of the system reaches the critical dirty threshold c , the oracle switches to ACID mode, and all following transactions are tagged ACID. To get back to BASE, the inconsistency level d_k has to drop to the recovery threshold r .

$$\text{BASE} \rightarrow \text{ACID} \quad \text{when} \quad d_k \geq c \quad (4.1)$$

$$\text{ACID} \rightarrow \text{BASE} \quad \text{when} \quad d_k \leq r \quad (4.2)$$

with the interval $[r, c]$ acting as the hysteresis band (Figure 4.7).

The oracle locks one mode for the whole system until the switch conditions are met. Every new transaction admission is tagged with whatever that mode is at that instant. Work that is already in flight keeps the mode it was admitted under, by the lifetime rules of Section 4.4. A mode change is therefore not an event that the running transactions experience.

The inconsistency level is the only metric the oracle uses, so every mode transition has a single clear cause. Figure 4.7 shows how the adaptive oracle behaves when the mode switch happens and which tag a transaction receives at each point of the run.

The two edges of the band are not symmetric. Past the critical dirty fraction, no further BASE work is admitted, and only the BASE transactions already in flight still add dirty resources, so the overshoot is small. The undershoot past the recovery threshold can be large. The oracle reads the dirty fraction only when a new root transaction is tagged, and in ACID mode, this happens rarely. Reconciliation keeps clearing the dirty set between two readings, so the reading that triggers the switch is often well below the recovery threshold. Section 5.7 gives more details.

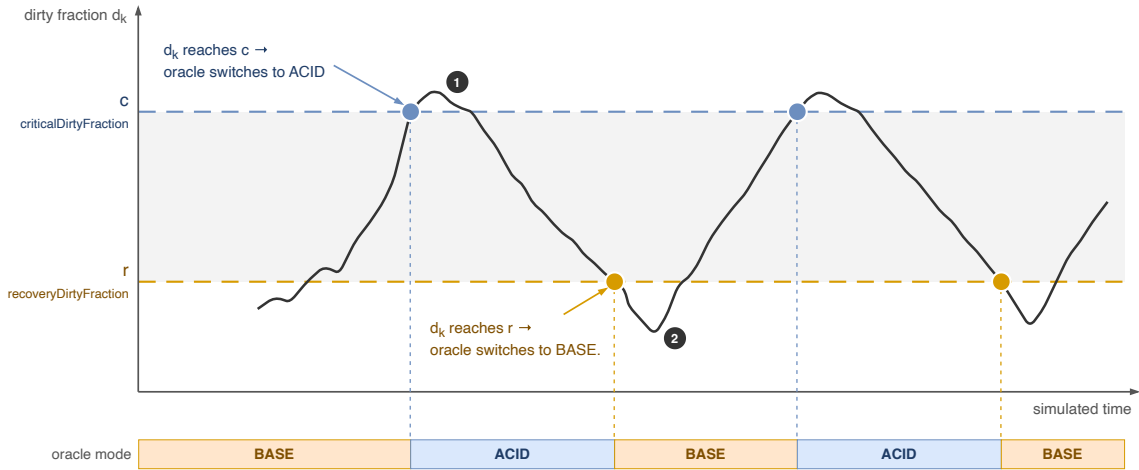


Figure 4.7.: The hysteresis band against a dirty-fraction trace.

- ① Small overshoot past the critical dirty threshold, caused by BASE work still in flight.
- ② Large undershoot past the recovery threshold, because the oracle reads the dirty fraction rarely in ACID mode.

The critical dirty and recovery thresholds. The oracle has two parameters. The critical dirty threshold, also called the inconsistency level threshold, is the fraction of resources that may be dirty before the oracle stops issuing BASE tags. It states in the configuration how much inconsistency the system is willing to accept.

The recovery threshold is the fraction the system must reconcile back down to before BASE resumes. The reconcilers work in both modes. What changes in ACID mode is that new transactions no longer add dirty resources, so the reconcilers can catch up. Both thresholds are fractions of the resources, and the recovery threshold r is configured directly rather than

derived from the critical dirty threshold c , subject to $0 \leq r \leq c$.

The two extremes of that range are worth naming, because they are policies and not just values. At $r = c$, the band has zero width, and the only anti-flap mechanism in the design is gone. The simulator accepts this configuration, but it is not a recommended setting, and no experiment in this thesis uses it. At $r = 0$, the rule says reconcile everything before resuming BASE, which differs from a small r , because an ACID visit then ends only when the dirty set is empty, so its length is set by the reconciler pool alone.

The oracle reads the inconsistency level at admission. The oracle reads the dirty fraction inside the call that assigns a mode. That is once per untagged root transaction, at its first admission attempt. Nothing else triggers a reading. The simulation tick does not force one, and no reading is taken while the system is progressing but admits nothing.

This ties the rate at which the oracle reads the dirty fraction to the rate at which new root transactions arrive. Under EQUILIBRIUM, the transaction population is held constant, so arrivals follow completions, and the reading rate of the oracle is about the throughput. When the adaptive oracle is in ACID mode, the throughput drops strongly, and so does the reading rate. Section 5.7 measures the consequences.

The reported measurements of the dirty fraction are not taken this way. The reading of the oracle is part of the system under study. The reported measurements come from a separate reader that runs once at the end of each simulated tick, regardless of mode or load. The percentiles of the dirty fraction in Chapter 5, such as p_{95} , are taken over these per-tick readings. Only the levels at a mode switch are readings of the oracle.

Reading d_k only at admission is still enough, because a reading matters only when a transaction has to be tagged. A reading once per tick would record the switch to BASE closer to r , so the measured undershoot would be smaller. It would not change the behavior of the system, because it would not let any BASE transaction start earlier. Under EQUILIBRIUM, a new root transaction is created only when an in-flight one finishes. In ACID mode, these transactions are long, and they hold the admission capacity while reconciliation clears the dirty set in the background. The first BASE transaction is therefore admitted at the same moment with either kind of reading.

Collision probability of ACID transactions and dirty resources. The arbiter makes an ACID transaction wait on every dirty resource it needs. Under the adaptive oracle, the dirty fraction stays close to the critical dirty threshold, so its size is roughly known. This allows us to model a useful quantity, the probability that one ACID transaction has to wait on a dirty resource. The cost comes from the arbiter and not from the choice of the tag, so it applies to the fixed-share oracle as well.

One parameter has to be explained before the derivation. The baseline value $L = 50$ (`maxTransactionResources`) is the maximum number of resources one transaction can touch. The simulator draws the resource count ℓ of each transaction uniformly from 1 to L . The derivation assumes that these resources are independent of each other and that each of them is dirty with probability d .

In this setup, a single resource is clean with probability $1 - d$. A transaction with ℓ resources

is clean only if every one of them is clean, so the probabilities multiply.

$$P_{\text{clean}}(d) = \underbrace{(1-d) \cdot (1-d) \cdots (1-d)}_{\ell \text{ resources}} = (1-d)^\ell \quad (4.3)$$

Blocking is the complementary event, in which at least one of the ℓ resources is dirty.

$$P_{\text{block}}(d) = 1 - P_{\text{clean}}(d) = 1 - (1-d)^\ell \quad (4.4)$$

Figure 4.8 plots this probability against the dirty fraction d from 0 to 0.12 for two cases. The first is the worst case $\ell = L = 50$. The second is the average over all transaction sizes ℓ from 1 to 50. The figure shows that blocking is not a rare event, but the normal case for an ACID transaction admitted while the dirty set is large. The dirty fraction is not always as high as $c = 0.10$. But even at the baseline recovery threshold $r = 0.05$, an average ACID transaction is blocked by a dirty resource with a probability of 65%. In addition, Equation 4.4 ignores the resources claimed by BASE parent transactions, which makes the probability even higher.

This paragraph shows the additional cost that the arbiter introduces in a hybrid run, whichever oracle issues the tags. ACID transactions now have to wait not only on the locks of other ACID transactions, but also on the resources claimed by BASE parents and on the dirty resources left by BASE bookkeeper transactions. Unfortunately, this is unavoidable because otherwise the isolation promise of ACID would be broken.

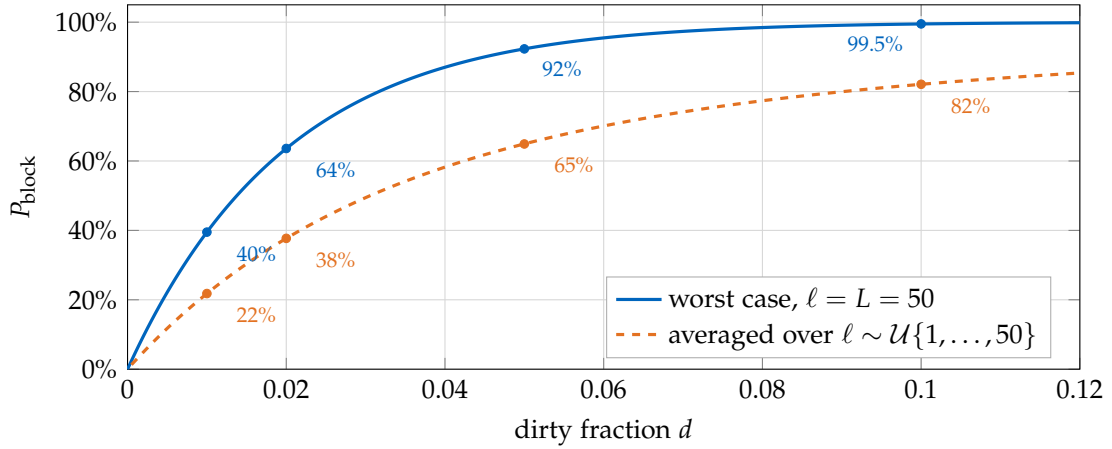


Figure 4.8.: Blocking probability against the dirty fraction.

A weighted average of the dirty fraction as a considered design. One design decision taken along the way was to smooth the inconsistency level before the oracle looks at it. The thresholds were not compared against the inconsistency level d_k directly, but against an exponentially weighted moving average (EWMA) $\bar{d}_k$ of it. At every reading, the average was updated with the smoothing factor α .

$$\bar{d}_k = \alpha d_k + (1 - \alpha) \bar{d}_{k-1}, \quad 0 \leq \alpha \leq 1, \quad \bar{d}_0 = d_0 \quad (4.5)$$

At $\alpha = 1$, the average is equal to the live value d_k , so there is no smoothing. The baseline smoothing factor was $\alpha = 0.2$. This means that a new reading d_k contributed only 20% to the average.

A filtered signal was expected to reduce noise and to avoid rapid mode switches. That decision was reversed, and the adaptive oracle now reads the live dirty fraction d_k at the moment of admission. The smoothing was removed for three reasons.

- **The dirty fraction is an exact value.** The dirty fraction is an exact number maintained by the simulator, and it has no measurement noise. The filter, therefore, had nothing to remove. It only delayed real changes of d_k . In addition, the anti-flap work was already done by the hysteresis band (Section 2.4).
- **The switch at the critical dirty threshold was unaffected.** In earlier runs with the filter, which are not reported in this thesis, the measured p_{95} of the dirty fraction stayed within about one percentage point of the critical dirty threshold at every setting of the smoothing factor. The BASE $\rightarrow$ ACID switch, therefore, behaved the same way with and without the filter.
- **It increased the undershoot of the recovery threshold.** The smoothing added a second lag to the ACID $\rightarrow$ BASE switch, on top of the gap between two readings described above. As a result, the system became much cleaner than configured before BASE resumed, and the recovery threshold had little effect.

The hybrid mode is designed to work with any oracle. The two oracles of this section are only two possible policies. The hybrid mode itself does not depend on how a tag is issued. The dispatchers, the arbiter, and the tagging rules work with any component that implements the `ModeOracle` interface. A new policy, therefore, only needs one new class, which is connected through the hybrid runtime assembler.

For example, other policies could focus on the throughput or on the length of the pending queue, or they could use a learned policy like the one used by Hybrid Transactional Replication [9]. Every such oracle also keeps the safety properties of Section 4.7, because they come from the arbiter and not from the choice of the tag.

4.7. Safety of the hybrid mode

RQ1 asks how a system can add a hybrid mode that adapts between ACID and BASE *safely*.

What safe means in this work.

- **S1. Transactions never corrupt each other's work.** A resource is never under an ACID lock and written by BASE at the same time, neither claimed by a BASE parent nor marked dirty by a BASE repair. This is the invariant of Section 4.5. It is the property that keeps the ACID promise. Otherwise, the results of one transaction could overwrite the work of another and corrupt the state of the system.

- **S2. No work is dismissed or forgotten.** Every transaction that is made to wait is looked at again, and keeps being retried for as long as it waits. Nothing that is parked or deferred is forgotten or silently dropped.
- **S3. Every run is free of deadlock.** The simulator never reaches a state in which work is waiting, but none of it can proceed.

Two things are not part of this definition on purpose. Safety here says nothing about how long a wait lasts. It also says nothing about fairness, so one piece of work can be delayed much longer than another. S2 promises only that the work is still there and is still retried, not that it runs soon.

S1 holds because of the shared arbiter design. In a pure run, each mechanism reads only its own records, and that is enough, because the records of the other mode are always empty. In a hybrid run, the records of both modes are in use at once, so each side has to consider the work of the other. The arbiter of Section 4.5 places a check on every path that uses a resource. There are four such paths. These are the ACID root at admission, the ACID bookkeeper when it adds locks, the BASE parent at admission, and the BASE repair at rule expansion (Section 4.5). Every transaction, therefore, verifies whether the resources it needs are available. If one of them is not, the transaction waits until it is able to proceed.

S2 holds because every waiting entry is looked at again. The two waiting structures avoid dismissing outstanding work in two different ways.

The ACID transactions registered as waiters are all woken as soon as the resource they wait on is available again. This works the same way for dirty and for claimed resources. The waiters are notified as soon as the event happens.

The deferred queues, on the other hand, are polled because the event that matters to them is not visible in the same way. Nothing tells a deferred entry that a lock is gone. Each deferred entry is therefore retried at fixed points of the simulation, which are described in Section 4.5. Nothing guarantees when a BASE transaction will be able to proceed, but every entry is retried at least once per simulation tick. As mentioned in the previous sections, polling fits BASE, since, by design, this mode aims only at eventual consistency.

The hybrid mode, therefore, either keeps a waiting transaction in a monitored structure, where it is notified on a specific event, or polls it systematically. No transaction is dropped or deleted while it waits.

S3 holds because no new wait cycles are introduced. A deadlock is a cycle of transactions in which each one holds something that the next one waits for. Figure 4.9 shows the three ways one transaction can end up waiting for another in a hybrid run, and only the first of them has a cycle. That first one comes from the base simulator, which already breaks it by aborting one group.

The two waiting mechanisms added by this thesis cannot close a cycle. A deferred BASE entry holds nothing, so no transaction can wait on it. An ACID waiter is different. A root parked at admission holds nothing either, but a bookkeeper parked in mid-execution keeps the locks of its group. That wait still cannot be part of a cycle, because what it waits on never

waits itself. A dirty resource is cleared by a repair that has already been created and needs no lock. A claim is dropped by a BASE parent that has already been admitted and never blocks again. Neither of them can wait on the parked group, so the wait always ends.

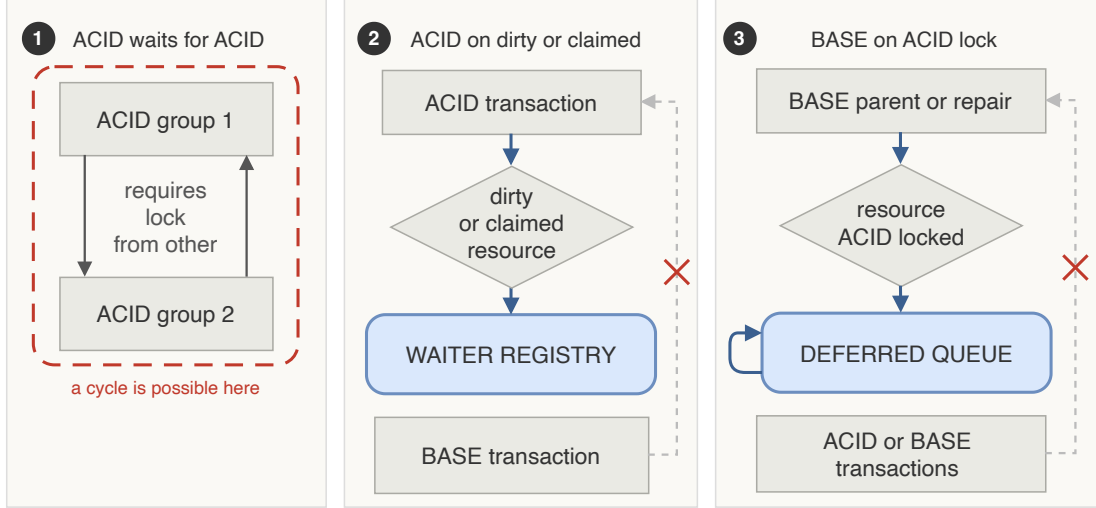


Figure 4.9.: The three kinds of waiting in a hybrid run.

- ① Received cycle, broken by cautious waiting.
- ② ACID waits on BASE work that never waits back.
- ③ Deferred BASE work holds nothing, so nothing waits on it.

As Figure 4.9 shows, the arbiter adds waiting but no new cycle. Two further points follow from that. First, the arbiter can add aborts. A group whose bookkeeper is parked in the waiter registry is blocked and keeps its locks. Under the first abort rule of Section 4.3, another ACID transaction that asks for one of those locks is aborted instead of being queued. Second, the rest of the cost of waiting shows up as time.

The known imprecisions do not break safety. The two imprecisions of Section 4.5 do not weaken the three properties, because both of them err in the same direction. Under C1, coarse lock granularity makes the BASE checks look at a whole bucket of resources instead of one. Work can then be deferred that did not have to be, which costs time but never breaks S1. Under C2, a deferred entry whose resource stays locked for a long time can be significantly delayed. That is a case of starvation rather than deadlock, so S2 and S3 still hold.

The properties hold by construction. The three properties are established by design and not by measurement. S1 holds because the four checks sit on the four paths that use a resource, and there is no fifth path. S2 and S3 hold because of the shape of the waiting that those checks produce. No experiment in this thesis was designed to test these properties directly.

5. Experiments

To evaluate the hybrid mode, we ran five experiments on the extended simulator. Section 5.1 first describes configurations shared by all of them. The first experiment answers how the hybrid mode compares with pure ACID and pure BASE (RQ2), the second experiment answers the question of inconsistency threshold configurability (RQ2), and the third experiment answers how the adaptive oracle performs compared to a static ACID/BASE split (RQ3). Safe adaptation between ACID and BASE (RQ1) is covered only partially by the experiments, since it is a design question rather than an experimental one. The fourth and fifth experiments extend the answer to (RQ2) and give further insight into the system, the critical dirty threshold, and how well this threshold holds under different loads. Section 5.7 takes a closer look at the recovery threshold. Section 5.8 closes the chapter with the threats to validity.

- **X1: Reconciler-processor sweep** (Section 5.2). This experiment varies the number of reconciler processors available in the system. It is used to measure how background reconciliation affects different semantic configurations in the simulator.
- **X2: Critical dirty fraction threshold sweep** (Section 5.3). This experiment sweeps the critical dirty threshold introduced in this thesis. It tests whether this threshold holds, meaning if the measured inconsistency fraction follows it.
- **X3: Fixed-share oracle ACID/BASE split sweep** (Section 5.4). This experiment studies how the system behaves when a fixed ACID/BASE split is configured at the start of the simulation. It compares this with the adaptive oracle, where the system chooses ACID or BASE at runtime.
- **X4: Semantic overlap sweep** (Section 5.5). This experiment sweeps the semantic overlap, which controls how many resources are shared between the models.
- **X5: Max number of resources per transaction sweep** (Section 5.6). This experiment sweeps the maximum number of resources a single transaction touches. In this way, it controls how many resources, and so how much work, one transaction brings.

5.1. Experimental setup

This section shows the baseline simulator configuration, described in Table 5.1. Every experiment starts from it and changes only the variables it sweeps. Each sweep is crossed with the number of reconciler processors. Most settings come from the table of simulation parameters and the default configuration of Zacouris and Acosta [1, Table 1]. The deviations

from it are listed below the table. Thresholds are given as fractions in the configuration, while the measured dirty fraction is reported in percent.

Table 5.1.: Baseline configuration.

Parameter	Baseline value	What it controls
Resources N	5000	Size of the data set
Models	10	Number of models
Semantic overlap σ	0.30	Information shared by models (X4)
IO processors	$30 - R$	IO capacity
CPU processors	$30 - R$	CPU capacity
Locks	5000	Available locks
Reconcilers R	1, 5, 10, 15	Reconciliation processors
Referential action	ON_CASCADE	Handling of cross-model repairs
Bookkeeper depth	5	How far a repair propagates
IO time	200 ms	IO work of one resource
CPU time	100 ms	CPU work of one resource
Lock IO time	200 ms	IO cost of one lock
Lock CPU time	100 ms	CPU cost of one lock
Step length	1 s	Simulated time of one step
Profile	EQUILIBRIUM	Constant number of transactions
In-flight transactions	30	Concurrent transactions
Resources per transaction L	50	Max. resources touched (X5)
Models per transaction	up to 4	Max. models reached
Critical threshold c	0.10	BASE $\rightarrow$ ACID switch (X2)
Recovery threshold r	0.05	ACID $\rightarrow$ BASE switch
Fixed ACID share p_{acid}	0.50	Share started in ACID (X3)
Duration	50 000 s	Simulated time
Seed pairs	5	Replicates per cell

Processors. The reconciler processors are not extra capacity. In BASE and HYBRID runs, the reconcilers are taken out of the 30 foreground processors, so the IO and CPU processors left for foreground work are 30 minus the number of reconcilers. Pure ACID runs use no reconcilers and keep all 30.

Workload. The simulator creates its own workload, so no external data set is used. The EQUILIBRIUM profile keeps the number of in-flight transactions constant. Whenever one transaction finishes, a new one is created at the next step.

Deviations from the reference configuration. Three new entries were added to the configuration, the critical dirty threshold, the recovery dirty threshold, and the fixed ACID probability. One value differs from the reference table.

The only deviation is that the number of simulated seconds was raised from 10 000 s to 50 000 s. The reason is that switches between ACID and BASE take time. Therefore, more simulated time is needed to let enough switches happen for a reliable comparison.

Seeds. The simulator provides the *transaction-sampling seed*. It fixes how many resources each transaction touches, how many models it reaches, and which models and resources it uses. The same seed also drives the draws of the fixed-share oracle.

The second seed, the *semantic-overlap seed*, fixes which resources are shared between the different models. The semantic-overlap seed, therefore, fixes the state of the multi-model resource pool.

Experiments implementation. For each experiment, a dedicated runner script was created. It configures the baseline variables and allows the selected variable to be swept. Every script is then run under the five seed pairs shared across all experiments. An experiment provides two types of results, the fixed-step simulation log and a CSV file in which each row represents one simulation. A row combines the configuration of that run with the different output metrics we want to study.

Metrics. Three values matter most for the evaluation.

- **Throughput** (`throughput_tps`). Root transactions completed per second of simulated time. Bookkeepers are not counted.
- p_{50} **dirty fraction** (`dirty_p50_pct`). The median share of resources that are inconsistent. This describes the typical state of the system.
- p_{95} **dirty fraction** (`dirty_p95_pct`). The 95th percentile of the same quantity. Within one run, inconsistency stays below this value for 95% of the simulated time. This describes the bad case, and it is the number that a threshold is supposed to limit.

The resource dirty fraction is sampled at the end of every simulator step, that is, once per simulated second. We call this value the measured dirty fraction (also inconsistency level), and its 95th percentile is the quantity limited in Equation 1.1.

Two further numbers describe the ACID part of the run.

- **ACID time share** (ϕ_{time}). How much of the run the system spent in ACID mode. The oracle timestamps every decision and sums the lengths of the intervals that were marked as ACID.
- **ACID completed share** (ϕ_{comp}). What share of the completed transactions were tagged as ACID.

5.2. X1: Reconciler-processor sweep

X1 answers the first part of RQ2, how the hybrid mode compares with pure ACID and pure BASE in throughput and inconsistency. It is the only sweep in which all three modes are started under the same conditions.

Experiment configuration. The sweep covers reconciler processors in $\{0, 1, 5, 10, 15, 20, 25\}$ for four configurations, pure ACID, pure BASE, HYBRID_ADAPTIVE, and HYBRID_FIXED_SHARE.

The fixed-share oracle runs with a fixed ACID share of 0.5.

Table 5.2 reports one row per configuration and number of reconciler processors. *Switch.* shows how many mode changes happened, and both $\text{BASE} \rightarrow \text{ACID}$ and $\text{ACID} \rightarrow \text{BASE}$ changes are counted. ϕ_{time} is the share of time the system spent in ACID mode, and ϕ_{comp} is the fraction of transactions completed with the ACID tag.

In Table 5.2, ACID gets a single row for all seven reconciler counts, since reconciler processors, by design, have no influence in pure ACID mode.

Processor configuration rule. Reconcilers are taken out of the 30 foreground processors (Section 5.1). At the end of the sweep, only 5 foreground processors are left.

Table 5.2.: X1: results.

R	Mode	Thr. (tps)	ϕ_{time}	ϕ_{comp}	Switch.	p_{50} (%)	p_{95} (%)
–	ACID (all R)	0.0418	1.0000	1.0000	0.0	n/a by construction	
0	BASE	1.9536	0.0000	0.0000	0.0	3.17	3.87
1	BASE	3.8685	0.0000	0.0000	0.0	29.50	29.64
5	BASE	3.3344	0.0000	0.0000	0.0	27.19	27.52
10	BASE	2.6681	0.0000	0.0000	0.0	23.38	23.93
15	BASE	2.0013	0.0000	0.0000	0.0	17.73	18.46
20	BASE	1.3343	0.0000	0.0000	0.0	8.10	8.91
25	BASE	0.6665	0.0000	0.0000	0.0	0.16	0.50
0	HYBRID_ADAPTIVE	1.9536	0.0000	0.0000	0.0	3.17	3.87
1	HYBRID_ADAPTIVE	0.0711	0.4998	0.1221	46.4	5.73	10.47
5	HYBRID_ADAPTIVE	0.2793	0.3290	0.1336	137.4	3.00	10.62
10	HYBRID_ADAPTIVE	0.3560	0.1974	0.1183	144.4	0.17	9.80
15	HYBRID_ADAPTIVE	0.4188	0.1377	0.1003	138.0	0.02	9.13
20	HYBRID_ADAPTIVE	1.3161	0.0016	0.0006	2.4	8.07	8.90
25	HYBRID_ADAPTIVE	0.6665	0.0000	0.0000	0.0	0.16	0.50
0	HYBRID_FIXED_SHARE	0.0877	0.6048	0.4986	2207.2	0.00	0.06
1	HYBRID_FIXED_SHARE	0.0844	0.6122	0.4985	2123.0	1.08	3.10
5	HYBRID_FIXED_SHARE	0.0880	0.5966	0.4983	2212.2	0.00	0.29
10	HYBRID_FIXED_SHARE	0.0877	0.5966	0.4987	2206.4	0.00	0.12
15	HYBRID_FIXED_SHARE	0.0883	0.5908	0.4982	2220.2	0.00	0.08
20	HYBRID_FIXED_SHARE	0.0935	0.5297	0.4985	2344.4	0.00	0.07
25	HYBRID_FIXED_SHARE	0.0986	0.5122	0.5007	2468.0	0.00	0.07

Validation against the base simulator. The BASE rows are compared with the received simulator [1] to check that the pure runs still behave as before. The reference reports that a few reconcilers push inconsistency up to the semantic overlap, and that more reconcilers lower inconsistency at the cost of throughput. Table 5.2 shows the same trade-off. The p_{50} inconsistency read from Figure 12a of the reference matches ours. With 15 reconcilers, the reference shows about 18%, and our run shows 17.7%. The small differences are most likely caused by our longer run of 50 000 s.

Edge case with zero reconciler processors. The row with zero reconcilers looks contradictory at first. Without reconciler processors, it combines a low dirty fraction with a high throughput. The reason is that repairs are queued on the foreground processors and run there once the transaction that caused them has committed [1, Section 4.5]. So the simulator has two cases.

- **No dedicated reconcilers.** Repairs are distributed over all 30 processors, so the number in flight is bounded by the pool rather than by the number of reconcilers.
- **Foreground processors converted.** One worker owns one repair until it finishes, so no more repairs are in flight at the same time than there are reconcilers. Each worker finishes one repair every two seconds because the reconciliation design separates IO and CPU work.

Pure ACID/BASE compared to HYBRID. Figure 5.1 shows that both hybrid configurations lie between pure ACID and pure BASE in throughput. In runs where the dirty fraction never rises to the critical dirty threshold, HYBRID_ADAPTIVE behaves like pure BASE.

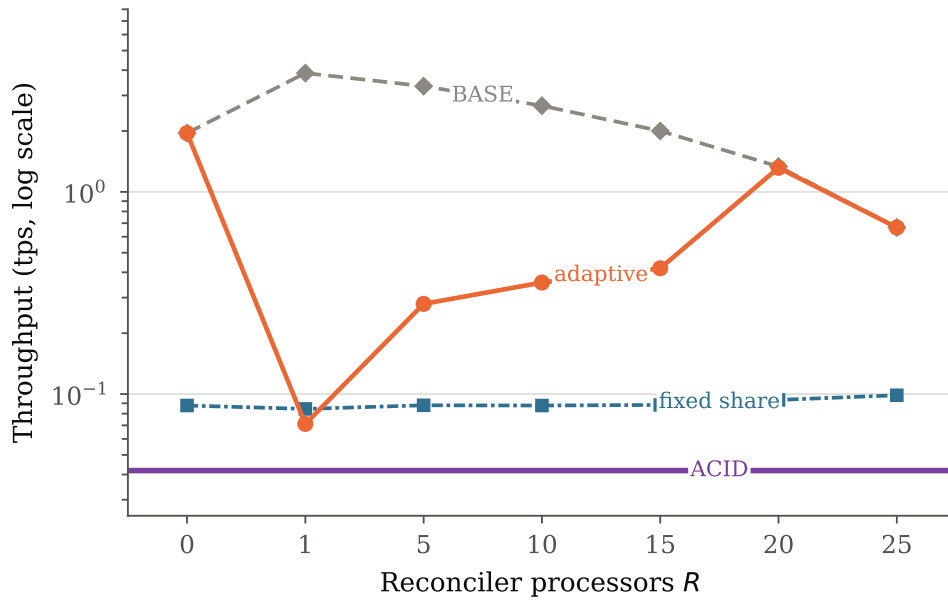


Figure 5.1.: X1: log-scaled throughput of all configurations.

Effect of reconciler processors on the fixed-share oracle. In Table 5.2, the small throughput improvement of HYBRID_FIXED_SHARE seems unexpected at first. More reconciler processors mean fewer foreground ones, so the throughput should decrease. What actually happens is that fewer foreground processors run fewer transactions at the same time, so fewer ACID transactions hold locks and abort. In addition, more reconciler processors clear the background work faster, which results in fewer ACID waits on dirty resources.

Findings

- **O1. None of the runs stall.** Every one of the 140 runs makes progress, and no run stalls completely. This gives a weak argument for RQ1, since no deadlock was encountered within

50 000 s.

- **O2. The adaptive oracle engages only from 1 to 15 reconcilers.** With 0 and with 20 or more reconciler processors, every adaptive run behaves like pure BASE.¹ The adaptive oracle starts in BASE mode, so in runs where the dirty fraction never reaches the configured threshold, the oracle never engages. Figure 5.2 compares the throughput of the adaptive oracle and pure BASE.

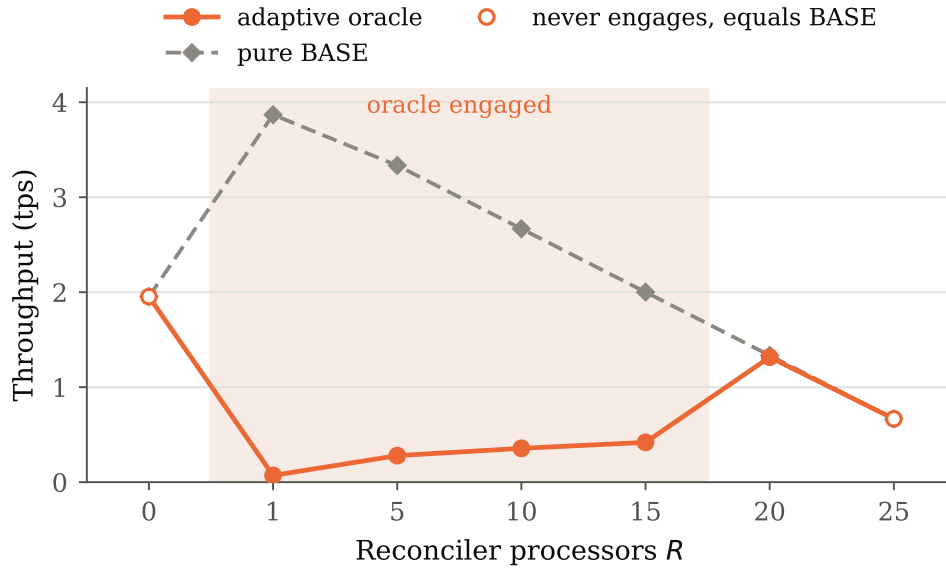


Figure 5.2.: X1: adaptive and pure BASE throughput.

- **O3. Hybrid throughput lands between ACID and BASE.** Figure 5.1 shows that both the adaptive and the fixed-share oracle are faster than pure ACID at every reconciler count. They are slower than pure BASE wherever the oracle is active, which is from 1 to 15 reconcilers for the adaptive oracle and at every reconciler count for fixed-share.
- **O4. The configured threshold bounds inconsistency, but the limit is not hard.** With 1 to 15 reconcilers, the adaptive p_{95} stays close to the critical dirty threshold of 10% and never exceeds it by more than 0.62 percentage points. The limit is not hard, since p_{99} reaches 11.9%, as Figure 5.3 shows. This is only a first check, and X2 studies the critical dirty threshold in detail (Section 5.3).

¹With 20 reconcilers, the oracle still switches mode, on average 2.4 times per run, and spends only 0.16% of the time in ACID. This difference is too small to matter, so from 20 reconciler processors on, the adaptive oracle is counted as BASE.

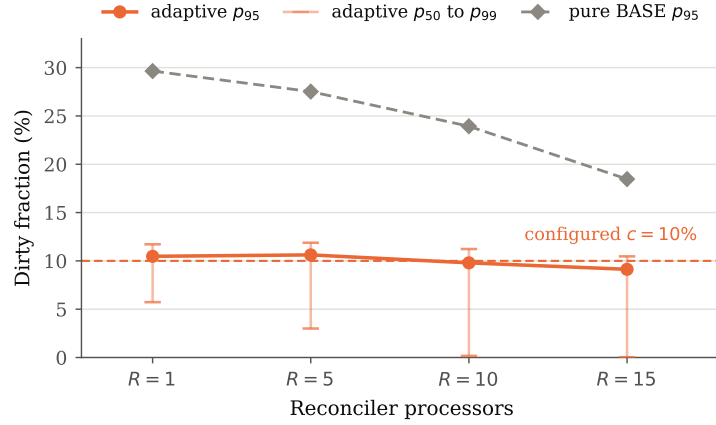


Figure 5.3.: X1: the dirty fraction when the adaptive oracle engages (O4).

5.3. X2: Critical dirty fraction threshold sweep

RQ2 asks whether the hybrid mode with the adaptive oracle makes it possible to control the inconsistency. X1 answers this question only partially, because it keeps the critical dirty threshold at 0.10. X2 varies this threshold directly and checks whether the measured dirty resource fraction follows it.

Experiment configuration. The experiment covers critical dirty thresholds of $\{0.02, 0.05, 0.10, 0.15, 0.20, 0.30\}$ and $\{1, 5, 10, 15\}$ reconciler processors, with all runs in HYBRID_ADAPTIVE.

The recovery threshold is held at half of the critical dirty threshold, $r = 0.5c$, which lets the hysteresis band scale with it. A fixed recovery threshold would cause an error for varying critical dirty fraction, since the simulator only accepts $\text{recovery threshold} \leq \text{critical dirty threshold}$.

Runs with reconciler counts outside $\{1, 5, 10, 15\}$ are left out, because X1 shows that the adaptive oracle behaves like pure BASE there.

One detail of this experiment is worth mentioning, which is the semantic overlap of 0.30. Only 30% of the 5000 resources carry a CPR, so, by design, a run can be at most 30% dirty, and this is a hard ceiling on the inconsistency.

Table 5.3 gives the throughput, the p_{95} of the measured dirty fraction, and the number of mode switches for each critical dirty threshold. A cell marked “deg.” is degenerate, which means that no mode switches happened and HYBRID_ADAPTIVE ran only in BASE mode. The mean ACID wait in Table 5.5 is the mean time an ACID transaction is blocked, counted from the moment it meets a dirty or claimed resource that it needs.

The critical dirty threshold sets the inconsistency level. Table 5.4 gives the value of $p_{95}(\text{dirty fraction}) - \text{critical dirty threshold}$, which is the distance from the configured critical dirty threshold in percentage points. A positive entry means that $p_{95}(\text{dirty fraction})$ overshoots the threshold, and a negative one means that the run stayed below it.

Table 5.3.: X2: experiment results.

c	$R = 1$			$R = 5$			$R = 10$			$R = 15$		
	Thr.	p_{95}	Switch.	Thr.	p_{95}	Switch.	Thr.	p_{95}	Switch.	Thr.	p_{95}	Switch.
0.02	0.0907	3.31	164	0.1885	3.11	259	0.2345	2.54	252	0.2650	1.96	225
0.05	0.0732	6.09	93	0.2308	6.23	172	0.2885	5.40	184	0.3319	4.68	174
0.10	0.0711	10.47	46	0.2793	10.62	137	0.3560	9.80	144	0.4188	9.13	138
0.15	0.0788	14.92	30	0.3310	14.90	109	0.4509	14.18	123	0.5915	14.10	116
0.20	0.0908	19.14	21	0.3994	19.20	85	0.5881	18.89	106		deg.	
0.30		deg.			deg.			deg.			deg.	

Table 5.4.: X2: threshold tracking.

Configured c	$R = 1$	$R = 5$	$R = 10$	$R = 15$	Range (pp)
0.02	+1.31	+1.11	+0.54	-0.04	1.35
0.05	+1.09	+1.23	+0.40	-0.32	1.55
0.10	+0.47	+0.62	-0.20	-0.87	1.49
0.15	-0.08	-0.10	-0.82	-0.90	0.82
0.20	-0.86	-0.80	-1.11	deg.	0.31
0.30	deg.	deg.	deg.	deg.	-

This is the central result of the thesis. The p_{95} dirty fraction is never more than 1.31 percentage points away from the configured critical dirty threshold, and the further a run is from the edges of the threshold range, the smaller this deviation is. In terms of Equation 1.1, the critical dirty threshold works as a configurable limit on the inconsistency level.

The inconsistency deviation changes from overshoot to undershoot. From Table 5.4, we see that the average deviation of p_{95} turns from positive to negative. It changes direction both when the critical dirty threshold grows and when more reconciler processors are added. Figure 5.4 helps to understand the cause. As the critical dirty threshold grows, more dirty resources are allowed, so ACID transactions wait longer to be admitted. While the ACID transactions wait, the background work keeps running. As a result, the foreground of the simulator slows down strongly, while the reconciler processors keep clearing the dirty resources.

A high number of completed ACID transactions lowers throughput. The throughput column of Table 5.3 is counterintuitive at first. By Equation 4.4, the probability that an ACID transaction is blocked by at least one dirty resource climbs fast as the critical dirty threshold increases. A small critical dirty threshold should therefore be the faster setting. The measurements show the opposite. With 5 or more reconcilers, the runs with a small critical dirty threshold are the slow ones.

The reason is that a small critical dirty threshold forces the system to tag more transactions as ACID, because it has to keep the inconsistency level low. The system has to complete more

slow work, and as a result, the throughput drops towards pure ACID. As the critical dirty threshold is relaxed, the fraction of completed transactions tagged ACID drops significantly, because the system can focus on fast BASE transactions (Figures 5.4 and 5.5).

We also see that the time spent in ACID stays almost flat for all runs with five or more reconciler processors, which is directly related to the mean ACID wait. The system still spends much time in ACID because each ACID period gets longer, but most of the throughput comes from completed BASE transactions.

Table 5.5.: X2: transaction mix.

R	c	ACID transactions			The run as a whole			
		Adm. A	Mean ACID wait (s)	ACID _{cmt}	BASE _{cmt}	ϕ_{time}	ϕ_{comp}	Thr. (tps)
1	0.02	2633	109.1	2162	2371	0.696	0.477	0.0907
1	0.05	1171	214.3	989	2671	0.625	0.270	0.0732
1	0.10	498	344.3	434	3123	0.500	0.122	0.0711
1	0.15	327	447.2	294	3645	0.434	0.075	0.0788
1	0.20	253	549.1	218	4321	0.372	0.048	0.0908
5	0.02	2731	23.2	2229	7197	0.332	0.237	0.1885
5	0.05	2652	43.1	2254	9286	0.309	0.195	0.2308
5	0.10	2136	67.8	1864	12099	0.329	0.134	0.2793
5	0.15	1612	88.7	1430	15120	0.320	0.086	0.3310
5	0.20	1298	107.3	1153	18816	0.308	0.058	0.3994
10	0.02	2709	11.7	2203	9522	0.239	0.188	0.2345
10	0.05	2689	21.1	2238	12188	0.203	0.155	0.2885
10	0.10	2487	34.1	2105	15695	0.197	0.118	0.3560
10	0.15	2160	45.5	1825	20722	0.201	0.081	0.4509
10	0.20	1859	55.5	1583	27824	0.195	0.054	0.5881
15	0.02	2695	7.0	2255	10997	0.170	0.170	0.2650
15	0.05	2644	14.0	2241	14353	0.149	0.135	0.3319
15	0.10	2463	23.7	2101	18839	0.138	0.100	0.4188
15	0.15	2091	31.6	1782	27791	0.130	0.060	0.5915

5. Experiments

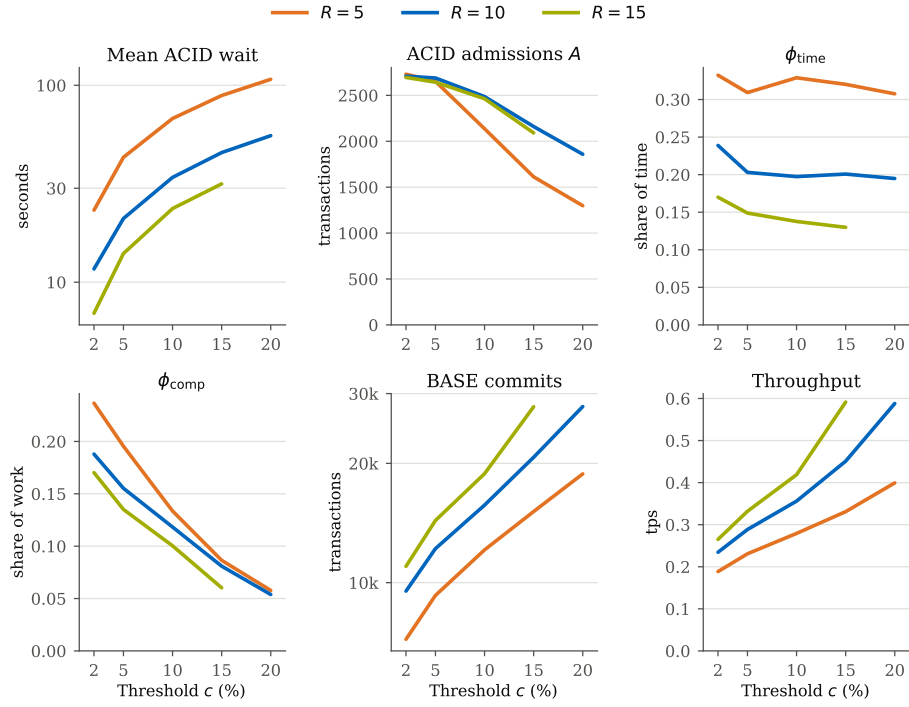


Figure 5.4.: X2: ACID and BASE work with 5 to 15 reconcilers.

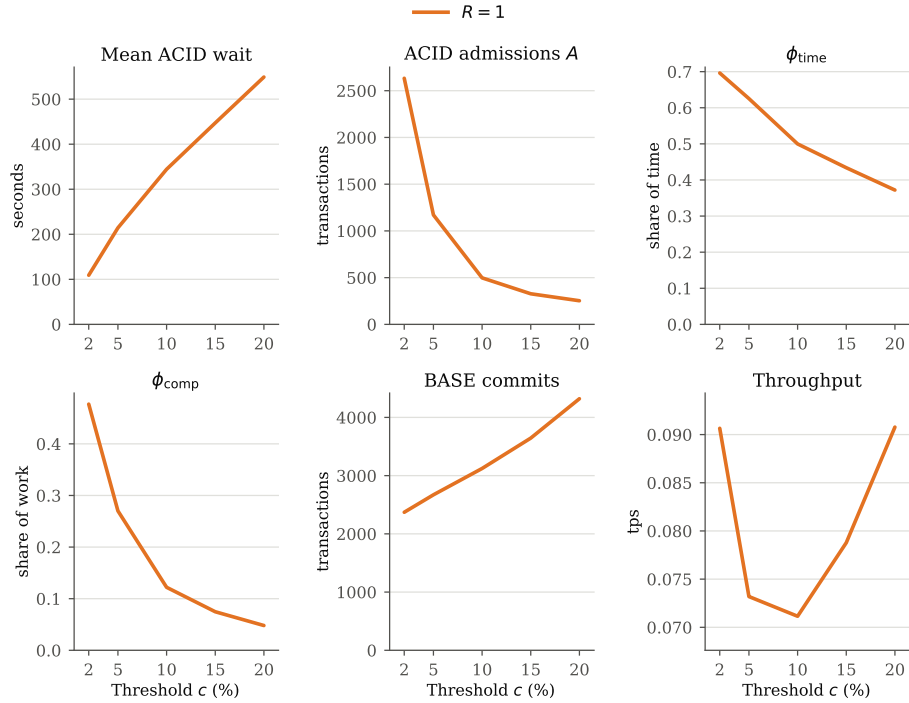


Figure 5.5.: X2: ACID and BASE work with one reconciler.

Findings

- **O5. The critical dirty threshold controls inconsistency.** The measured p_{95} dirty fraction is at most 1.31 percentage points away from the configured critical dirty threshold. This answers the part of RQ2 about the configurability of inconsistency. It extends O4, because X1 only covered a critical dirty threshold of 0.10, while X2 sweeps a range of values.
- **O6. A high threshold disables the adaptive oracle.** O5 holds only while the oracle actually engages. The oracle stops engaging once the inconsistency that BASE produces on its own stays below the configured critical dirty threshold, so it never switches to ACID. The configurable range of RQ2, therefore, has an upper limit, which is set by how dirty the system becomes on its own.
- **O7. A higher threshold moves work from ACID to BASE.** As the critical dirty threshold increases, the adaptive oracle allows more BASE transactions, and the share of completed ACID transactions drops significantly.

5.4. X3: Fixed-share oracle ACID/BASE split sweep

The fixed-share oracle (Section 4.6) allows users to set the probability of starting a transaction in ACID mode. It was introduced as a baseline for the adaptive oracle. In fixed-share, the split is set at the beginning, and afterwards, the system uses no information about its internal state. The third experiment answers the third research question, which asks whether the adaptive oracle performs better than a static split decided at the start.

Experiment configuration. The ACID share takes the values {0.05, 0.10, 0.15, 0.25, 0.50}. Each of them is run with {1, 5, 10, 15} reconciler processors. The Switch. column of Table 5.6 counts mode changes from ACID to BASE and back.

Every run in this thesis is executed under EQUILIBRIUM, which is a stationary load. The comparison below is therefore made in a setting that is not ideal for the adaptive oracle.

The ACID share can set the inconsistency level. Table 5.6 shows how the ACID share and the number of reconcilers allow inconsistency to be controlled under a static workload, and the core principles from the previous experiments hold here as well. The two settings act on different parts of the system.

The ACID share acts on what the system is asked to do. Under EQUILIBRIUM, the number of in-flight transactions is held constant (Section 5.1), so increasing the ACID share replaces fast BASE transactions with slow ACID ones. Throughput falls, fewer transactions are admitted per second, and less dirtying work enters the system.

The number of reconcilers acts on how fast the dirty set is cleared. More reconcilers drain the same dirty set faster, so the number of dirty resources is smaller overall. The rows with one reconciler show what happens when this second setting is too small to keep up. A single reconciliation processor is unable to clear the high volume of dirty resources, which increases the ACID wait time.

5. Experiments

Table 5.6.: X3: results.

R	p_{acid}	Thr. (tps)	ACID _{cmt}	BASE _{cmt}	ϕ_{time}	ϕ_{comp}	Switch.	p_{50} (%)	p_{95} (%)
1	0.05	0.1036	238	4940	0.3341	0.0461	491.2	9.50	21.73
	0.10	0.0777	369	3516	0.3225	0.0950	713.8	6.80	14.73
	0.15	0.0720	512	3086	0.3460	0.1423	909.4	5.38	11.16
	0.25	0.0728	883	2757	0.4206	0.2426	1366.4	3.87	7.73
	0.50	0.0844	2104	2117	0.6122	0.4985	2123.0	1.08	3.10
5	0.05	0.4228	1030	20107	0.1289	0.0488	2004.6	8.23	20.25
	0.10	0.3171	1576	14277	0.1864	0.0994	2865.8	4.60	13.72
	0.15	0.2650	1955	11296	0.2344	0.1476	3338.2	2.06	10.32
	0.25	0.1800	2219	6780	0.3392	0.2467	3360.4	0.10	3.49
	0.50	0.0880	2191	2208	0.5966	0.4983	2212.2	0.00	0.29
10	0.05	0.6225	1520	29602	0.0743	0.0489	2921.2	5.37	17.04
	0.10	0.4233	2105	19059	0.1456	0.0995	3811.2	0.78	11.15
	0.15	0.3052	2271	12992	0.2120	0.1488	3868.4	0.07	5.31
	0.25	0.1810	2229	6821	0.3387	0.2464	3377.0	0.00	1.00
	0.50	0.0877	2186	2201	0.5966	0.4987	2206.4	0.00	0.12
15	0.05	0.7324	1780	34838	0.0711	0.0486	3418.8	2.30	14.09
	0.10	0.4582	2274	20634	0.1387	0.0993	4119.4	0.10	7.37
	0.15	0.3090	2299	13150	0.2154	0.1489	3913.0	0.00	1.91
	0.25	0.1817	2237	6847	0.3392	0.2464	3388.2	0.00	0.38
	0.50	0.0883	2199	2217	0.5908	0.4982	2220.2	0.00	0.08

Taken together, the share of transactions started as ACID controls the throughput and the inconsistency of the system, while the number of reconcilers is an additional setting that controls how fast the inconsistency is drained.

Matching inconsistency for throughput comparison. The two oracles are configured in different ways. By setting the critical dirty threshold to 0.10, we configure the adaptive oracle to limit inconsistency near 10%. Setting the ACID share to 0.10 makes the fixed-share oracle start one transaction in ten in ACID mode. No pair of settings makes the two runs comparable by configuration, so the runs are matched on their output, the measured inconsistency.

An exact match rarely exists, because the fixed-share experiment was swept at only five probabilities. Table 5.7 groups the adaptive and fixed-share runs by the number of reconciler processors. The adaptive runs are the sweep of the critical dirty threshold from the second experiment (Table 5.3), and the fixed-share runs are the sweep of the ACID share from Table 5.6. Inside each group, the table places every adaptive run between the two fixed-share runs on either side of it. These two fixed-share neighbors are the inconsistency levels reached under a specific ACID share. The match distance to the nearer of the two is defined as

$$\text{distance} = \min\left(\frac{p_{95}^{\text{adaptive}}}{p_{95}^{\text{clean}}}, \frac{p_{95}^{\text{dirty}}}{p_{95}^{\text{adaptive}}}\right)$$

Table 5.7.: X3: matched throughput.

R	c	adaptive	cleaner neighbor		dirtier neighbor		To nearest	Thr. (tps)		
		p_{95}	p_{acid}	p_{95}	p_{acid}	p_{95}	Dist.	adaptive	fixed	Ratio
1	0.02	3.31	0.50	3.10	0.25	7.73	1.07	0.0907	0.0839	1.08
	0.05	6.09	0.50	3.10	0.25	7.73	1.27	0.0732	0.0769	0.95
	0.10	10.47	0.25	7.73	0.15	11.16	1.07	0.0711	0.0721	0.99
	0.15	14.92	0.10	14.73	0.05	21.73	1.01	0.0788	0.0784	1.01
	0.20	19.14	0.10	14.73	0.05	21.73	1.14	0.0908	0.0940	0.97
5	0.02	3.11	0.50	0.29	0.25	3.49	1.12	0.1885	0.1690	1.12
	0.05	6.23	0.25	3.49	0.15	10.32	1.66	0.2308	0.2141	1.08
	0.10	10.62	0.15	10.32	0.10	13.72	1.03	0.2793	0.2697	1.04
	0.15	14.90	0.10	13.72	0.05	20.25	1.09	0.3310	0.3362	0.98
	0.20	19.20	0.10	13.72	0.05	20.25	1.05	0.3994	0.4058	0.98
10	0.02	2.54	0.25	1.00	0.15	5.31	2.09	0.2345	0.2253	1.04
	0.05	5.40	0.15	5.31	0.10	11.15	1.02	0.2885	0.3072	0.94
	0.10	9.80	0.15	5.31	0.10	11.15	1.14	0.3560	0.3960	0.90
	0.15	14.18	0.10	11.15	0.05	17.04	1.20	0.4509	0.5260	0.86
15	0.02	1.96	0.15	1.91	0.10	7.37	1.02	0.2650	0.3102	0.85
	0.05	4.68	0.15	1.91	0.10	7.37	1.57	0.3319	0.3846	0.86
	0.10	9.13	0.10	7.37	0.05	14.09	1.24	0.4188	0.5298	0.79
	0.15	14.10	0.05	14.09	—	—	1.00	0.5915	0.7324	0.81

The fixed-share throughput in Table 5.7 is interpolated linearly between the two neighboring rows of Table 5.6.

One candidate run with a critical dirty threshold of 0.20 and 10 reconcilers is left out. Its p_{95} is higher than that of the dirtiest fixed-share run with 10 reconcilers, so it is dropped rather than extrapolated.

The resulting throughput ratios range from 0.79 to 1.12, and they depend on the number of reconcilers. With 1 and 5 reconcilers, the two oracles are close, and a few adaptive rows are faster. With 10 and 15 reconcilers, the adaptive oracle is slower in almost every row.

Consider only the ten closest rows, where the nearest neighbor lies within 12%. In all ten rows, the two oracles reach almost the same share of completed ACID transactions at matched inconsistency. In different ways, they control the same quantity, which is the ACID share of the completed work. The time spent in ACID mode, however, differs, and in seven of the ten rows the adaptive oracle has the higher value.

One weakness of the adaptive oracle compared to the fixed-share. The adaptive oracle switches to ACID mode at the moment when the dirty fraction reaches the critical dirty threshold, which is when the system has already reached the highest allowed level of inconsistency. The dirtier the system gets, the longer ACID transactions have to wait. Table 5.8 shows that at every reconciler count, a larger share of the adaptive ACID transactions is blocked on a dirty or BASE-claimed resource, and their mean blocked wait is longer. While

fixed-share starts a mix of ACID and BASE transactions at all times, the adaptive oracle starts all transactions in ACID, while the dirty fraction travels from the critical dirty threshold back down to the recovery threshold. That likely leads to more ACID transactions in flight at the same time, which compete for locks and wait on dirty resources.

Table 5.8.: X3: ACID blocking and aborts.

R	c	nearest p_{acid}	Adm. A		Blocked (%)		Mean ACID wait (s)		Aborted (%)	
			adaptive	fixed	adaptive	fixed	adaptive	fixed	adaptive	fixed
1	0.02	0.50	2633	2544	54.6	41.6	109.1	99.4	38.2	41.6
	0.05	0.25	1171	1595	72.2	51.2	214.3	179.1	21.1	29.8
	0.10	0.15	498	667	85.5	72.0	344.3	294.3	8.6	11.7
	0.15	0.10	327	406	92.7	82.0	447.2	393.4	4.2	5.3
	0.20	0.05	253	319	99.0	86.8	549.1	484.1	3.0	3.7
5	0.02	0.25	2731	2686	60.6	38.1	23.2	20.5	19.8	22.9
	0.05	0.15	2652	2509	78.2	49.9	43.1	35.7	15.9	16.3
	0.10	0.15	2136	2219	89.2	66.0	67.8	56.6	9.2	10.5
	0.15	0.10	1612	1671	96.3	77.1	88.7	79.4	5.9	5.9
	0.20	0.05	1298	1254	99.6	85.0	107.3	99.3	3.9	3.7
10	0.02	0.15	2709	2676	60.7	36.9	11.7	10.1	14.9	15.8
	0.05	0.15	2689	2627	77.1	49.4	21.1	18.3	12.6	10.2
	0.10	0.10	2487	2452	87.2	61.4	34.1	28.6	9.4	7.1
	0.15	0.05	2160	2050	91.6	72.2	45.5	40.7	6.7	4.3
15	0.02	0.15	2695	2652	55.5	39.1	7.0	6.4	12.0	9.2
	0.05	0.10	2644	2612	73.6	47.1	14.0	11.5	9.6	7.1
	0.10	0.10	2463	2423	84.7	59.8	23.7	19.9	7.7	4.6
	0.15	0.05	2091	2001	90.0	76.2	31.6	29.3	4.5	2.7

Implications for a real software system. If we know that the workload will be static, we can set the share of transactions that are started in the slow ACID mode. This increases the throughput compared to pure ACID while still controlling the inconsistency level of the system.

As a result, fixed-share is cheaper to run, since it does not read the internal system state, and it always performs better than pure ACID. It allows the inconsistency to be controlled within a certain range, and if the tuning effort is acceptable, the fixed-share split between ACID and BASE is a viable option.

Findings

- **O8. The ACID transaction share controls inconsistency.** Under constant load, the ACID share together with the number of reconcilers reaches the inconsistency levels of the adaptive oracle, so the decision does not need to be based on the internal system state.

- **O9. The adaptive oracle falls behind as the number of reconciler processors increases.**

At matched inconsistency, the two oracles are close with 1 and 5 reconcilers. With 10 and 15 reconcilers, the adaptive oracle is slower. For RQ3, this means that, under a stationary load, choosing the mode from the internal state does not outperform a fixed split that was tuned for each configuration.

5.5. X4: Semantic overlap sweep

Semantic overlap describes how many resources are shared between the models of a multi-model system (Section 2.3). This experiment gives more insight into RQ2, especially into the part where the critical dirty threshold acts as the control for the level of inconsistency. The semantic overlap sets the highest dirty fraction the system can reach, so it is important to see how the critical dirty threshold behaves under different values of the semantic overlap.

Experiment configuration. The sweep covers semantic overlap values in {0.00, 0.05, 0.10, 0.20, 0.30, 0.50, 1.00} against {1, 5, 10, 15} reconciler processors, with a critical dirty threshold of 0.10 and a recovery threshold of 0.05. A cell in which the oracle never switches is marked degenerate. The column d_k^{switch} is the reading of the oracle at the switch. It is taken only at the moments when the oracle switches to ACID mode and is averaged over all switches of the run. *Aborts* is the share of finished attempts that aborted. Aborts are counted as events, so one transaction can abort more than once.

The adaptive oracle does not engage for all semantic overlap values. Thirteen of the 28 cells in Table 5.9 behave exactly like a pure BASE run, because the adaptive oracle never switches to ACID. The reason is that the semantic overlap sets a hard ceiling on how much inconsistency is possible by design, while the oracle engages only where its reading of the dirty fraction reaches the threshold. At a semantic overlap of 0.10, the ceiling equals the critical dirty threshold of 10%. With one reconciler, the dirty fraction is already pinned near the ceiling, with a p_{95} of 9.62%. In the cells where the oracle does engage, more semantic overlap raises the background cost of a transaction. As a result, throughput falls, aborts rise, and the system spends more time in ACID.

Figure 5.6 plots the dirty fraction against semantic overlap, with the configured critical dirty threshold as the dashed line. In the shaded range from a semantic overlap of 0.20 to 0.30, the p_{95} dirty fraction lands where it was configured. At a semantic overlap of 0.50, the threshold starts to drift, and as semantic overlap approaches 1.00, the threshold no longer holds, with a p_{95} of up to 22.51%.

Why the p_{95} (dirty fraction) overshoots the threshold. The trigger works, but the limit on p_{95} does not. The oracle still engages where it was configured to engage, as the column with the reading at the switch (d_k^{switch}) in Table 5.9 shows. In every engaged cell of the sweep, the oracle switches to ACID mode just above the configured critical dirty threshold of 10%.

What breaks the threshold is the work already in flight. A tag is fixed at admission (Section 4.4), so a mode switch affects only the transactions admitted after it and not the 30

Table 5.9.: X4: results.

σ	R	p_{50} (%)	p_{95} (%)	d_k^{switch} (%)	Switch.	ϕ_{time}	ϕ_{comp}	Thr. (tps)	Aborts
0.00	1	0.00	0.00	–	deg.	0.0000	0.0000	3.8685	0.000
0.00	5	0.00	0.00	–	deg.	0.0000	0.0000	3.3344	0.000
0.00	10	0.00	0.00	–	deg.	0.0000	0.0000	2.6681	0.000
0.00	15	0.00	0.00	–	deg.	0.0000	0.0000	2.0013	0.000
0.05	1	4.50	4.62	–	deg.	0.0000	0.0000	3.8685	0.000
0.05	5	2.14	2.43	–	deg.	0.0000	0.0000	3.3344	0.000
0.05	10	0.08	0.22	–	deg.	0.0000	0.0000	2.6681	0.000
0.05	15	0.04	0.15	–	deg.	0.0000	0.0000	2.0013	0.000
0.10	1	9.50	9.62	–	deg.	0.0000	0.0000	3.8685	0.000
0.10	5	7.16	7.47	–	deg.	0.0000	0.0000	3.3344	0.000
0.10	10	3.20	3.66	–	deg.	0.0000	0.0000	2.6681	0.000
0.10	15	0.14	0.35	–	deg.	0.0000	0.0000	2.0013	0.000
0.20	1	5.52	10.04	10.08	50.8	0.4236	0.0878	0.1347	0.031
0.20	5	2.45	9.98	10.11	157.0	0.2822	0.0989	0.4986	0.043
0.20	10	0.20	9.46	10.09	159.2	0.1601	0.0764	0.6814	0.038
0.20	15	7.54	8.23	–	deg.	0.0000	0.0000	2.0013	0.000
0.30	1	5.73	10.47	10.15	46.4	0.4998	0.1221	0.0711	0.086
0.30	5	3.00	10.62	10.18	137.4	0.3290	0.1336	0.2793	0.092
0.30	10	0.17	9.80	10.19	144.4	0.1974	0.1183	0.3560	0.094
0.30	15	0.02	9.13	10.14	138.0	0.1377	0.1003	0.4188	0.077
0.50	1	6.31	11.46	10.21	35.6	0.5925	0.1694	0.0303	0.177
0.50	5	4.39	11.47	10.31	102.4	0.3893	0.1679	0.1242	0.164
0.50	10	0.56	10.87	10.41	104.8	0.2344	0.1504	0.1634	0.180
0.50	15	0.03	9.67	10.37	106.6	0.1767	0.1349	0.1841	0.168
1.00	1	10.79	22.51	10.73	11.4	0.7720	0.2941	0.0069	0.303
1.00	5	4.32	17.79	10.58	29.2	0.4341	0.2639	0.0228	0.374
1.00	10	0.56	16.02	10.73	35.6	0.2880	0.2145	0.0303	0.355
1.00	15	0.00	14.53	10.67	39.2	0.2367	0.1925	0.0345	0.327

that are already running. Those transactions keep writing, and every write to an overlapped resource spawns a bookkeeper that inherits the BASE tag of its root. The higher the semantic overlap, the more of this background work each transaction creates. At a semantic overlap of 1.00, every resource has a CPR, so the writes of each root transaction cascade into bookkeepers that spawn further bookkeepers.

At full overlap, this cascade looks endless, but the simulator bounds it with the bookkeeper depth, which is 5 in every run of this chapter (Table 5.1). A bookkeeper at that depth creates no children, so each chain stops there.

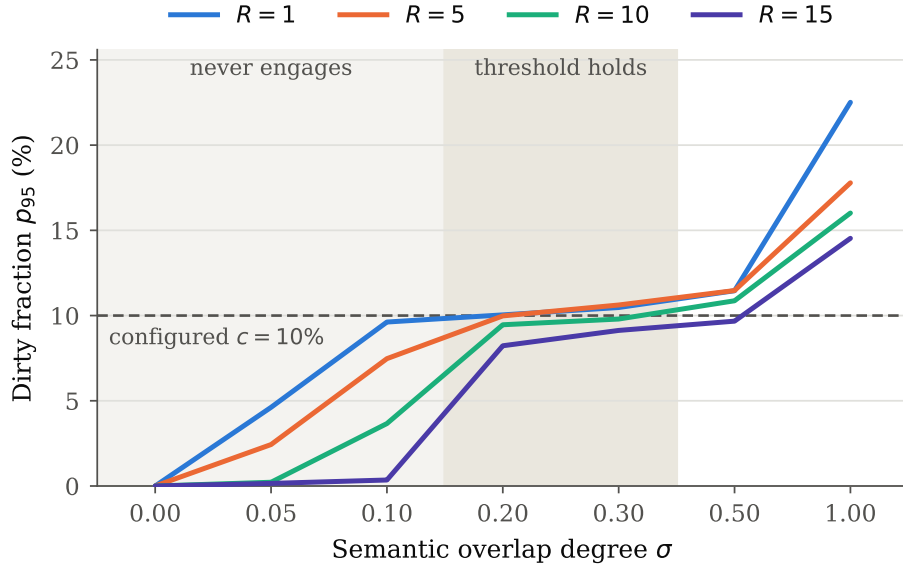


Figure 5.6.: X4: the measured tail against semantic overlap.

Findings

- **O10. The critical dirty threshold does not hold for all semantic overlap values.** The oracle needs enough overlap to fire at all. It also needs the overlap to stay small enough that one write does not dirty a large share of the resources in the background. Figure 5.6 shows that the critical dirty threshold holds only for semantic overlap values from 0.20 to 0.30. Below this range, every cell is a pure BASE run. Above it, a single write reaches so far that the transactions already admitted keep marking resources dirty in the background.
- **O11. In-flight BASE transactions overflow the critical dirty threshold.** The more the models overlap, the more background work every write creates. This background work comes from transactions that were tagged BASE before the switch, and they keep adding new bookkeepers even after the adaptive oracle switches to ACID.

5.6. X5: Max number of resources per transaction sweep

The maximum transaction size (`maxTransactionResources`) is the largest number of resources a single transaction may touch. Each transaction draws its size uniformly from 1 to this maximum and spreads those resources over up to four of the ten models.

Experiment configuration. The sweep covers maximum transaction sizes of {5, 10, 25, 50, 100, 200} resources against {1, 5, 10, 15} reconciler processors, with a critical dirty threshold of 0.10, a recovery threshold of 0.05, and a semantic overlap of 0.30. In Table 5.10, *Lat.* is the mean transaction latency from creation to commit.

Table 5.10.: X5: results.

L	R	p_{50} (%)	p_{95} (%)	p_{99} (%)	Switch.	Thr. (tps)	Lat. (s)	Aborts
5	1	6.94	9.71	9.96	84.2	0.6032	27.2	0.002
10	1	6.81	9.73	10.04	74.0	0.3316	57.3	0.005
25	1	6.22	9.88	10.46	54.8	0.1381	159.1	0.021
50	1	5.73	10.47	11.72	46.4	0.0711	340.5	0.086
100	1	5.43	11.51	13.51	40.8	0.0360	675.0	0.223
200	1	4.59	13.91	17.69	29.0	0.0180	1225.7	0.433
5	5	6.92	9.72	9.98	383.0	2.9429	5.7	0.000
10	5	6.73	9.76	10.09	330.2	1.6028	12.2	0.002
25	5	5.42	10.12	10.82	234.8	0.6603	33.3	0.015
50	5	3.00	10.62	11.88	137.4	0.2793	85.5	0.092
100	5	0.77	11.18	13.45	78.8	0.1093	234.2	0.296
200	5	0.05	10.84	14.84	46.2	0.0434	592.8	0.524
5	10	6.22	9.69	10.00	634.0	5.5296	3.2	0.000
10	10	5.16	9.71	10.14	512.4	2.9288	6.7	0.001
25	10	1.97	9.82	10.59	268.8	0.9496	22.0	0.015
50	10	0.17	9.80	11.23	144.4	0.3560	65.1	0.094
100	10	0.00	9.49	12.14	77.6	0.1397	182.7	0.277
200	10	0.00	8.32	13.32	40.2	0.0505	519.6	0.511
5	15	5.37	9.62	9.94	513.2	7.6077	2.7	0.000
10	15	3.50	9.53	10.01	458.6	3.6942	6.0	0.000
25	15	0.46	9.41	10.16	250.0	1.1263	19.9	0.008
50	15	0.02	9.13	10.47	138.0	0.4188	55.7	0.077
100	15	0.00	8.34	10.88	68.2	0.1545	165.0	0.257
200	15	0.00	7.37	11.53	37.0	0.0590	452.7	0.473

Transactions with a large pool of resources break the critical dirty threshold. For maximum sizes up to 50 resources, the measured p_{95} is at most 0.87 percentage points away from the configured critical dirty threshold of 10% at every reconciler count. Above a maximum size of 50, the threshold starts to break, and the measured p_{95} dirty fraction either overshoots or undershoots it, depending on the number of reconciler processors. Figure 5.7 shows that the critical dirty threshold holds only while transactions stay small.

The main reason for this deviation from the critical dirty threshold is the cost of a large transaction. With more resources, a transaction collides with other transactions more often and creates more background work. Table 5.10 shows that transaction latency grows with the maximum transaction size, so a transaction takes longer to commit. As a result, throughput and admissions drop, and so does the number of switches. Transactions most likely also wait longer in the queues, although only end-of-run queue lengths are recorded. The number of reconciler processors then decides the direction of the deviation. With few reconcilers, dirty resources take longer to clear, which causes the overshoot. With 10 or more reconcilers, transactions wait long, and throughput is low, so the reconciler processors clear more dirty resources in the background, and the p_{95} undershoots the critical dirty threshold.

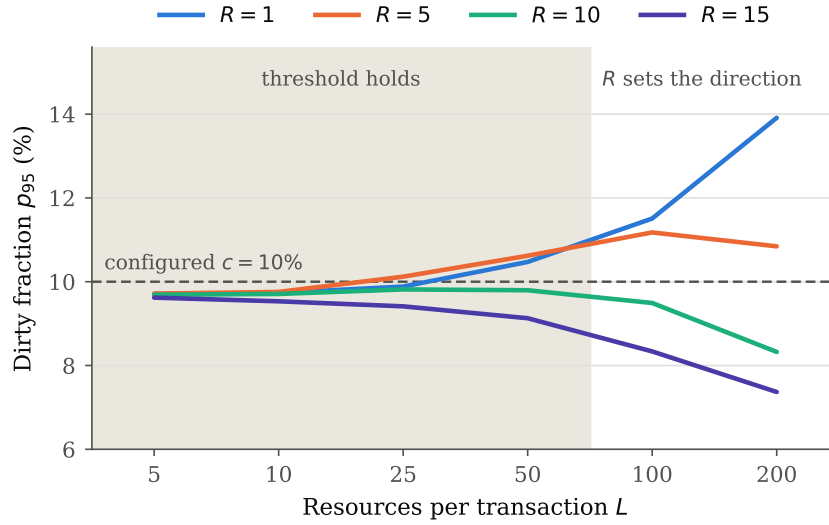


Figure 5.7.: X5: the measured tail against transaction size.

Findings

- **O12. Transaction size loads the foreground.** Large transactions take much longer to process and collide far more often, so the low throughput comes from waiting and from wasted work. Semantic overlap, in contrast, loads the reconciler because BASE resolves bookkeepers with dedicated reconciler processors.
- **O13. Reconciler processor count decides overshoot or undershoot of inconsistency.** The number of reconciler processors sets the direction. With few reconciler processors, the inconsistency level overshoots, because a switch binds only the transactions admitted after it, while the heavy ones already running keep dirtying resources. With enough reconciler processors, it undershoots, because heavy transactions wait for locks and deliver less work.

5.7. The effective recovery threshold

This section reports no new experiment and belongs to research question 2 about inconsistency configuration. It takes a closer look at the runs of the X2 critical dirty threshold sweep (Section 5.3) with a critical dirty threshold of 0.10 and a recovery threshold of 0.05. The oracle enters ACID when the dirty fraction reaches the critical dirty threshold and returns to BASE once it has fallen back to the recovery threshold. The previous sections reported only how the dirty fraction relates to the critical dirty threshold, while this section shows where the runs leave ACID.

Table 5.11 gives the median and the 95th percentile of the oracle readings taken at the moment of a switch. The ACID visit column is the mean length of one ACID visit, and the Drain column is the time that reconciliation alone needs to clear the dirty set from the critical dirty threshold down to the recovery threshold.

The switch from BASE to ACID happens close to the configured threshold at both p_{50} and p_{95} . The switch from ACID to BASE does not, and its median lies below the configured recovery threshold of 5%. Figure 5.8 draws the exit switch against the configured recovery threshold, together with the shortfall of each point.

Table 5.11.: Where the oracle switches.

R	BASE $\rightarrow$ ACID		ACID $\rightarrow$ BASE		ACID visit	Drain	Check interval	
	p_{50} (%)	p_{95} (%)	p_{50} (%)	p_{95} (%)	(s)	(s)	txn_{ACID} (s)	txn_{BASE} (s)
1	10.10	10.43	3.92	4.90	1075	500	57.5	8.0
5	10.14	10.47	3.84	4.89	240	100	8.8	2.8
10	10.14	10.48	3.05	4.90	137	50	4.7	2.6
15	10.10	10.37	2.61	4.76	100	33	3.3	2.3

Why the switch from ACID to BASE undershoots the recovery threshold. This problem was already mentioned in the previous experiments and in the approach description (Section 4.6), and here we explain it in more detail. The oracle reads the inconsistency level only once per root transaction, at its first admission attempt. Under EQUILIBRIUM, a new root transaction is created only when an in-flight one finishes. In ACID mode, the 30 in-flight transactions are slow because they wait for locks and for dirty resources. While all of them are stuck, no transaction finishes, no new one arrives, and no reading is taken, so between two readings, the value the oracle holds is out of date. The check interval columns in Table 5.11 show this, because at every reconciler count, the oracle reads less often in ACID mode than in BASE mode. A reading once per tick would record the exit close to the recovery threshold, but the first BASE transaction would still be admitted only when an ACID transaction finishes (Section 4.6).

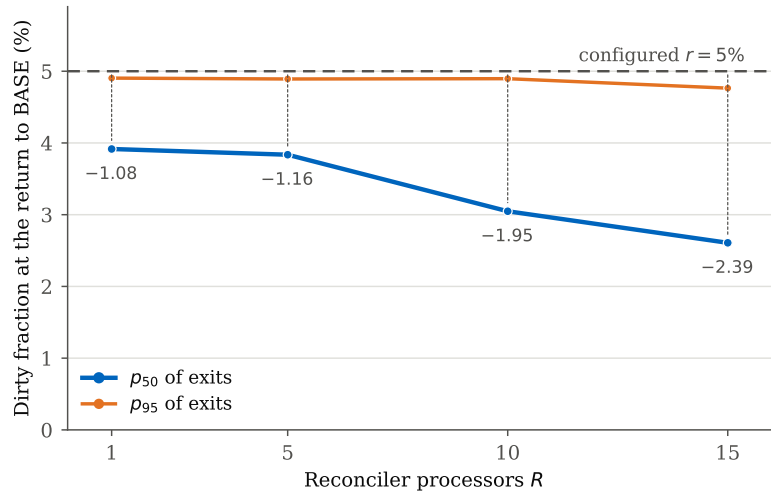


Figure 5.8.: Where the oracle returns to BASE.

5.8. Threats to validity

The results above hold under the conditions of this chapter. The following points limit how far they can be generalized.

- **Stationary load only.** Every run uses the EQUILIBRIUM profile, so the load never changes (Section 2.2). This is the setting in which feedback helps least.
- **Simulated execution.** The simulator treats all models as one entity, so a lookup of the arbiter has no latency and never fails (Section 4.5).
- **Five seeds per cell.** The results move little between seeds, but five runs give an indication rather than a confidence interval.
- **Hidden dirty resources.** A deferred repair is not in the dirty set until it is created (Section 4.5). The measured dirty fraction can therefore be lower than the real inconsistency, most of all in ACID mode.
- **Interpolated X3 comparison.** The fixed-share oracle is sampled at five values of the ACID share. Table 5.7 reads its throughput between measured points, and some of these points are far apart.

6. Conclusions and Future Work

This chapter summarizes the work, describes the lessons learned from combining ACID and BASE in one system, and lists open questions for future work.

6.1. Summary of the work

This thesis focused on whether ACID and BASE can run together in one multi-model system. To address this, the simulator of Zacouris and Acosta [1] was extended with a hybrid mode, in which a dedicated mechanism, the *mode oracle*, tags each transaction as ACID or BASE, and a shared-resource arbiter keeps the two semantics from corrupting each other's work. We built two mode oracles, an adaptive oracle driven by the dirty fraction and a fixed-share oracle, and compared their performance.

In the evaluation, the hybrid mode was faster than pure ACID and slower than pure BASE. The dirty fraction stayed close to the configured critical dirty threshold, but this held only for a range of workloads, and under constant load, the adaptive oracle did not outperform a fixed split. The hybrid mode, therefore, does not make the trade-off between ACID and BASE cheaper, but it allows different system parameters to be configured.

6.2. Lessons learned

ACID and BASE cannot simply run side by side, because without a resource management layer such as the arbiter, they corrupt the state of shared resources.

Sharing resources costs performance in both modes. ACID transactions now additionally wait on resources that are claimed by BASE or dirty, and BASE parent transactions wait on ACID locks. As a result, the hybrid mode cannot reach the throughput of a theoretical combination of ACID and BASE.

Once the resource management layer is in place, any oracle can be used without changes to the rest of the system, but every value it controls will be paid for with throughput.

Because the tag is fixed for the whole life of a transaction, transactions already in flight after a switch push the dirty fraction past the thresholds. Changing the mode of these transactions would allow more precise control.

6.3. Future work

- **Varying load.** Repeat the oracle comparison under a load that changes during a run, where feedback is expected to help most.

- **Real multi-model system.** Model the latency and failures of arbiter lookups, and validate the hybrid mode against the Vitruvius case studies.
- **Stronger statistics.** Run more seeds per cell and a finer grid of fixed ACID shares, and count the targets of deferred repairs as dirty.
- **Transaction-aware oracle.** Predict how heavy a transaction is from its resources and the updates it triggers, and send heavy work to BASE.
- **Other oracle policies.** React to the trend of the dirty fraction, read it on a fixed time interval, or learn the policy as HybML does [9].

A. Use of AI Tools

In line with the TUM AI Strategy, I disclose the use of generative AI tools in this thesis. I used Claude by Anthropic, both as a chat assistant and through Claude Code, for the following tasks.

- **Learning the codebase.** I received the simulator from earlier work, and the tool helped me understand its structure and code faster.
- **Literature search.** The tool helped me find related work. I read all cited sources myself and checked that their content matches the way they are described in this thesis.
- **Chart scripts.** The tool helped me write the scripts that plot the experiment results. I reviewed the scripts and checked the charts against the experiment data.
- **LaTeX tables.** The tool helped me move the results from the experiment sheets into LaTeX tables. I checked all values against the original data.
- **Proofreading.** The tool helped me correct grammar and spelling and improve the readability of the text.

I reviewed all output produced with the help of AI tools, and I take full responsibility for the content of this thesis.

Bibliography

- [1] Z. Zacouris and M. Acosta. “Modeling ACID and BASE Semantics for Cross-Model Consistency-Preserving Updates”. Unpublished manuscript. 2026.
- [2] Z. Zacouris and M. Acosta. “Simulating a Transactional Server for Multi-Model Systems”. In: *Proceedings of the VLDB Endowment* 18.12 (2025), pp. 5395–5398. doi: 10.14778/3750601.3750680.
- [3] D. R. Ries and M. R. Stonebraker. “Locking granularity revisited”. In: *ACM Transactions on Database Systems* 4.2 (1979), pp. 210–227. doi: 10.1145/320071.320078.
- [4] D. Pritchett. “BASE: An ACID Alternative”. In: *ACM Queue* 6.3 (2008), pp. 48–55.
- [5] W. Vogels. “Eventually consistent”. In: *Communications of the ACM* 52.1 (2009), pp. 40–44.
- [6] H. Klare, M. E. Kramer, M. Langhammer, D. Werle, E. Burger, and R. Reussner. “Enabling consistency in view-based system development — the Vitruvius approach”. In: *Journal of Systems and Software* 171 (2021), p. 110815.
- [7] M. Hsu and B. Zhang. “Performance evaluation of cautious waiting”. In: *ACM Transactions on Database Systems* 17.3 (1992), pp. 477–512. doi: 10.1145/132271.132275.
- [8] S. Dandamudi and S.-L. Au. “Locking granularity in multiprocessor database systems”. In: *Proceedings of the Seventh International Conference on Data Engineering (ICDE 1991)*. IEEE, 1991, pp. 268–277. doi: 10.1109/ICDE.1991.131474.
- [9] T. Kobus, M. Kokociński, and P. T. Wojciechowski. “Hybrid Transactional Replication: State-Machine and Deferred-Update Replication Combined”. In: *IEEE Transactions on Parallel and Distributed Systems* 29.7 (2018), pp. 1499–1514. doi: 10.1109/TPDS.2018.2796079.
- [10] M. Couceiro, P. Romano, and L. Rodrigues. “PolyCert: Polymorphic Self-Optimizing Replication for In-Memory Transactional Grids”. In: *Middleware 2011*. Vol. 7049. Lecture Notes in Computer Science. Springer, 2011, pp. 309–328.
- [11] M. Couceiro, P. Ruivo, P. Romano, and L. Rodrigues. “Chasing the Optimum in Replicated In-Memory Transactional Platforms via Protocol Adaptation”. In: *43rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2013)*. 2013.
- [12] C. Xie, C. Su, M. Kapritsos, Y. Wang, N. Yaghmazadeh, L. Alvisi, and P. Mahajan. “Salt: Combining ACID and BASE in a Distributed Database”. In: *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, 2014.

- [13] C. Li, D. Porto, A. Clement, J. Gehrke, N. M. Prego, and R. Rodrigues. “Making Geo-Replicated Systems Fast as Possible, Consistent when Necessary”. In: *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. USENIX Association, 2012, pp. 265–278.
- [14] T. Kraska, M. Hentschel, G. Alonso, and D. Kossmann. “Consistency Rationing in the Cloud: Pay only when it matters”. In: *Proceedings of the VLDB Endowment 2.1* (2009), pp. 253–264. DOI: 10.14778/1687627.1687657.
- [15] S. Sidhanta, W. M. Golab, S. Mukhopadhyay, and S. Basu. “OptCon: An Adaptable SLA-Aware Consistency Tuning Framework for Quorum-Based Stores”. In: *16th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid 2016)*. 2016, pp. 388–397.
- [16] C. Pu and A. Leff. “Replica Control in Distributed Systems: An Asynchronous Approach”. In: *Proceedings of the 1991 ACM SIGMOD International Conference on Management of Data*. 1991, pp. 377–386.
- [17] K.-L. Wu, P. S. Yu, and C. Pu. “Divergence Control for Epsilon-Serializability”. In: *Proceedings of the Eighth International Conference on Data Engineering (ICDE 1992)*. 1992, pp. 506–515.
- [18] H. Yu and A. Vahdat. “Design and Evaluation of a Continuous Consistency Model for Replicated Services”. In: *4th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2000)*. 2000, pp. 305–318.